

UNIVERSITÀ DEGLI STUDI DI TRIESTE

Dipartimento di Matematica, Informatica e Geoscienze



Corso di Laurea Triennale in Intelligenza Artificiale e Data Analytics

Prova Finale

Un sistema di supporto alla decisione clinica neuro-simbolico per la verifica della rimborsabilità delle Note AIFA

Laureando

Francesco Zamar

Relatore

Prof. Andrea De Lorenzo

Anno Accademico 2025/2026

Sommario

La verifica della rimborsabilità di un farmaco soggetto a Nota AIFA è oggi un'attività manuale che il medico prescrittore compie consultando regole testuali, condizioni cliniche del paziente e tabelle di esclusione. È un'attività ad alto rischio di errore, ad alto costo cognitivo e con conseguenze sia economiche, per il Servizio Sanitario Nazionale, sia cliniche, quando una controindicazione viene trascurata.

In questa tesi si descrivono la progettazione, l'implementazione e la valutazione di un sistema di supporto alla decisione clinica (*Clinical Decision Support System*, CDSS) neuro-simbolico per la verifica della rimborsabilità relativa a quattro Note AIFA: N01 (inibitori di pompa protonica nella gastroprotezione cronica), N13 (statine nella dislipidemia), N66 (antinfiammatori non steroidei) e N97 (anticoagulanti orali diretti nella fibrillazione atriale non valvolare). L'architettura separa nettamente la decisione di rimborsabilità, che è interamente affidata a un rule engine deterministico ispezionabile, dalla generazione della spiegazione clinica, che è invece prodotta da un *Large Language Model* (LLM) di taglia accessibile (Llama 3.1 8B Q4_K_M) inserito in una pipeline di *Retrieval-Augmented Generation* (RAG) sulle Note ufficiali AIFA. Il sistema garantisce per costruzione che ogni decisione si appoggi a una catena di regole esplicite e che ogni claim della spiegazione sia citato dai passaggi recuperati.

La valutazione è stata condotta su un insieme di riferimento di 122 casi clinici annotati manualmente, stratificato per Nota e per classe decisionale, ed è stata strutturata attorno a sette domande di ricerca, ciascuna ancorata a una metrica primaria. Il rule engine raggiunge una Macro F1 di 1,0000 e un pass rate del 100% sui 122 casi di riferimento, con intervallo di confidenza al 95% costruito mediante bootstrap e Wilson: un valore che va letto come misura di coerenza interna fra la codifica delle regole e l'annotazione dei casi (svolte dallo stesso autore), non come validità clinica esterna. Le baseline si fermano a una Macro F1 di 0,2408 (solo LLM) e di 0,3152 (LLM con retrieval ma senza rule engine). Il retrieval ottiene un Recall@10 di 0,9068 e un *Mean Reciprocal Rank* (MRR) di 1,0000. La pipeline LLM produce spiegazioni a hallucination rate nullo, citation coverage del 100% e Citation F1 di 0,9988. Il composito ALES (*Aggregated LLM Evaluation Score*), ispirato alla

letteratura sulla valutazione dei sistemi RAG, vale 0,6810 (mediana 0,6874, $n = 122$). Il test di idempotenza e di robustezza ai casi limite restituisce pass rate del 100%.

Il lavoro mostra che la separazione fra decisione simbolica ed esplicazione neurale è una strategia praticabile per costruire sistemi CDSS verificabili clinicamente: i fallimenti del componente generativo sono confinati a un layer non decisionale, mentre la sicurezza prescrittiva è garantita dalla logica trivalente sul layer simbolico. La tesi discute apertamente le limitazioni residue, in particolare la parziale auto-referenza dell'insieme di riferimento e l'incomparabilità diretta del composito ALES fra versioni successive del framework di valutazione.

Indice

Sommario	1
Elenco delle figure	6
Elenco delle tabelle	8
Elenco degli algoritmi	9
Introduzione	10
1 Il contesto clinico delle Note AIFA	13
1.1 Sistemi di supporto alla decisione clinica	13
1.2 La regolamentazione AIFA della rimborsabilità	14
1.3 Le quattro Note oggetto di studio	15
1.3.1 N01: gastroprotezione cronica	15
1.3.2 N13: ipolipemizzanti nella dislipidemia	16
1.3.3 N66: antinfiammatori non steroidei	17
1.3.4 N97: anticoagulanti orali nella fibrillazione atriale	17
1.4 Il problema della verifica della rimborsabilità	18
2 Fondamenti	20
2.1 Sistemi a regole e reasoning simbolico	20
2.1.1 Logica di Kleene a tre valori	21
2.2 Recupero e generazione condizionata	23
2.2.1 Architettura di un sistema RAG	23
2.2.2 Chunking e indicizzazione	24
2.2.3 Embedding, similarità densa e reranking	24
2.3 Approcci neuro-simbolici	26
2.4 Valutazione di sistemi RAG e CDSS	27

2.4.1	Metriche di retrieval	28
2.4.2	Metriche di fedeltà	29
2.4.3	Qualità della risposta, compositi e statistica	29
3	Architettura del sistema	32
3.1	Requisiti e separazione neuro-simbolica	32
3.2	Architettura complessiva	35
3.3	Il layer simbolico: rule engine	36
3.3.1	Modello dei dati e schema YAML	36
3.3.2	Tre famiglie di regole e logica Kleene	37
3.4	Il layer esplicativo: pipeline RAG	38
3.4.1	Ingestione e chunking	38
3.4.2	Modello di embedding, indice e retriever	38
3.4.3	Generazione e vincolo di fedeltà	39
3.5	Contratto API e flusso end-to-end	40
4	Implementazione	43
4.1	Stack tecnologico e organizzazione del codice	43
4.2	Rule engine: implementazione	44
4.2.1	Schema YAML delle regole	44
4.2.2	Tipi di regole e fasi di valutazione	45
4.2.3	Implementazione della logica Kleene	46
4.2.4	Algoritmo di valutazione	46
4.3	Pipeline RAG: ingestione e recupero	46
4.3.1	Ingestione del corpus normativo	46
4.3.2	Recupero e reranking	48
4.4	Orchestratore, riproducibilità e demo	48
4.4.1	Endpoint pubblici	50
4.4.2	Riproducibilità e cleanroom	50
4.4.3	Suite di test e demo interattiva	51
5	Impianto della valutazione	53
5.1	L'insieme dei casi di riferimento	53
5.2	Protocollo sperimentale e riproducibilità statistica	54
5.3	Le sette domande di ricerca	55

6	Le sette domande di ricerca	56
6.1	RQ1: Il rule engine decide come l'annotatore?	56
6.2	RQ2: Il retriever trova chunk rilevanti?	57
6.3	RQ3: L'LLM è fedele alle fonti?	58
6.4	RQ4: Il sistema è robusto a variazioni dell'input?	59
6.5	RQ5: Il sistema neuro-simbolico è meglio dei baseline?	59
6.6	RQ6: La motivazione clinica è verificabile dal medico?	61
6.7	RQ7: Il sistema è clinicamente utile e safety-compliant?	62
6.8	Sintesi delle risposte	63
6.9	Analisi per Nota	64
6.10	Limitazioni residue	64
7	Conclusioni	67
7.1	Sintesi dei risultati e contributi	67
7.2	Limitazioni del sistema	68
7.3	Lavori futuri	69
A	Riproducibilità della valutazione	70
	Bibliografia	72

Elenco delle figure

3.1	Deployment view dell'architettura a componenti. Le frecce continue rappresentano il flusso online durante la valutazione (REST/HTTP tra processi separati; filesystem per ChromaDB e PDF); la freccia tratteggiata indica la fase offline di ingestione: i PDF normativi vengono estratti, segmentati e indicizzati nelle quattro collezioni ChromaDB [1].	35
3.2	Flusso della pipeline RAG dalla richiesta al testo di risposta. Il Rule Engine viene invocato per primo e la sua decisione è iniettata nel prompt prima del contesto documentale. Il controllo dell'invariante di sicurezza in uscita dalla generazione garantisce che la spiegazione testuale non contraddica mai la decisione deterministica.	42
6.1	Macro F1 sui 122 casi di riferimento. Il sistema neuro-simbolico, grazie al rule engine deterministico, raggiunge 1,0000; la baseline LLM con retrieval si ferma a 0,3152, quella con retrieval e reranker (senza rule engine) a 0,2759 e quella con solo LLM degenera alla classe maggioritaria (0,2408). Risposta a RQ5.	61
6.2	Distribuzione del punteggio ALES sui 122 casi. I casi con tutti e quattro i componenti (media 0,7008) si concentrano nell'intervallo 0,7–0,8; i casi parziali (media 0,6419) sono più dispersi. La media complessiva è 0,6810. Risposta a RQ7.	63
6.3	Evoluzione del punteggio ALES medio attraverso le versioni del framework di valutazione. La discesa da 0,9787 a 0,6810 non riflette un peggioramento del sistema, ma la progressiva correzione di errori di misurazione (de-tautologizzazione delle metriche, completamento di QAEval su tutti i 122 casi) che gonfiavano artificialmente il punteggio.	66

Elenco delle tabelle

6.1	Performance del rule engine sui 122 casi di riferimento: report per-classe e Macro F1. Numero di support e accuratezza completa sulla diagonale della matrice di confusione.	57
6.2	Performance del retriever su 122 casi di riferimento: Recall@10 e MRR calcolati sui <i>rule_id</i> di riferimento, Citation F1 calcolata sui passaggi effettivamente citati nella spiegazione finale prodotta dall'orchestrator.	58
6.3	Confronto Macro F1 fra il sistema completo e quattro baseline alternative su 122 casi di riferimento. La baseline a classe maggioritaria assegna a tutti i casi la classe più frequente dell'insieme di riferimento (RIMBORSABILE). La baseline LLM+RAG+reranker replica il retrieval a due stadi del sistema completo (l'unico componente assente è il rule engine) e isola quindi il contributo del layer simbolico. La Macro F1 delle baseline è calcolata sulle tre classi emettibili dall'LLM; i 5 casi ROUTED, mai predicibili dalle baseline, restano nell'insieme e ne penalizzano la precisione: il confronto è conservativo a loro favore.	60
6.4	Metriche per la verificabilità clinica della spiegazione: copertura delle citazioni a livello di caso, copertura dei claim e QAEval F1 sulle micro-domande generate.	62
6.5	Composito ALES e suoi quattro componenti, aggregati su 122 casi (deviazione standard fra parentesi). Il componente ContextualUtility è calcolato solo sugli 81 casi per cui il giudice RAGAS restituisce le metriche di contesto.	63
6.6	Sintesi delle sette domande di ricerca: per ogni RQ (<i>Research Question</i>) la metrica primaria, il valore osservato sull'insieme di riferimento di 122 casi e il giudizio binario. I valori si riferiscono alla run overnight 2026-05-13.	64

6.7	Distribuzione per Nota e per classe decisionale dei 122 casi di riferimento, con l'esito della verifica del rule engine. Nessun caso è classificato fuori dalla classe annotata (matrice di confusione diagonale per ciascuna Nota); i report per caso sono in <code>evaluation/results/per_case_reports/</code>	65
-----	--	----

Elenco degli algoritmi

1	Valutazione di un caso clinico (<code>evaluate</code>)	47
2	Indicizzazione di una nota AIFA (<code>indicizza_nota</code>)	49

Introduzione

L'Agenzia Italiana del Farmaco (AIFA) regola la rimborsabilità di un sottoinsieme rilevante di farmaci attraverso le cosiddette *Note AIFA*, prescrizioni testuali che fissano criteri di inclusione, esclusione ed eccezione subordinati alle caratteristiche cliniche del paziente. Una sola nota può coinvolgere indici di rischio cardiovascolare, livelli sierici di vari analiti, presenza simultanea di più diagnosi e cronologie terapeutiche, e la decisione finale (se il farmaco è a carico del Servizio Sanitario Nazionale o del cittadino) dipende dalla congiunzione di tutte queste condizioni. La verifica avviene tipicamente in fase di prescrizione, quando il medico ha tempo limitato e deve consultare il testo della nota in parallelo all'anamnesi. È un'attività a basso valore aggiunto cognitivo ma ad alto costo di attenzione: un errore in più o in meno si traduce in farmaco non rimborsato a carico del paziente, in segnalazioni amministrative al medico o, nei casi peggiori, in un danno clinico quando una controindicazione viene trascurata.

I sistemi di supporto alla decisione clinica (*Clinical Decision Support Systems*, CDSS) nascono proprio per ridurre questo carico. Nella loro forma più diffusa codificano le regole in un motore deterministico e presentano al medico l'esito di una verifica binaria: è lo stato dell'arte per il controllo delle Note AIFA nei principali software gestionali italiani, ed è efficace finché le regole sono stabili e in numero contenuto. Restano però due limiti che pesano proprio nel momento della prescrizione. Il primo è la spiegazione: un esito *non rimborsabile* restituito senza motivazione costringe il medico, di fronte a una decisione che giudica controintuitiva, a risalire da solo alla condizione che ha bloccato la prescrizione, riaprendo il problema cognitivo che il sistema dichiarava di risolvere. Il secondo è la verificabilità: chi deve controllare il sistema (l'azienda sanitaria, l'ente regolatore, lo stesso autore) ha bisogno di ispezionare l'intera catena che porta dalla regola al testo della nota, e un esito binario non offre questa tracciabilità.

I *Large Language Model* (LLM) degli ultimi anni sembrano lo strumento naturale per colmare entrambi i limiti, soprattutto se combinati con tecniche di *Retrieval-Augmented Generation* (RAG): un modello generativo può articolare in linguaggio naturale il motivo di una decisione e citare i passaggi della nota da cui quel motivo

deriva. Affidare però a un LLM la decisione di rimborsabilità sarebbe imprudente sul piano clinico. Le garanzie di sicurezza richieste a un sistema prescrittivo non sono compatibili con un componente generativo: per quanto bassa sia la probabilità di errore su un dominio chiuso e documentato come quello delle Note AIFA, il suo output non è verificabile a priori contro una specifica scritta.

La tesi propone una via intermedia, di impianto neuro-simbolico, fondata su una separazione netta di responsabilità. La decisione di rimborsabilità è affidata in modo esclusivo a un componente simbolico deterministico, che traduce le Note AIFA in regole esplicite e ispezionabili: è qui, e solo qui, che risiedono le proprietà di sicurezza del sistema. Al modello generativo è riservato il solo compito di spiegare: riceve la decisione già presa e i passaggi delle Note pertinenti, e li riformula in un testo clinicamente leggibile, vincolato a citare le fonti che gli sono state fornite. La sicurezza della decisione è così garantita per costruzione; la qualità della spiegazione diventa invece una questione empirica, ed è la sua misura rigorosa il cuore del lavoro.

Il sistema è costruito e valutato su quattro Note AIFA (N01 per gli inibitori di pompa protonica nella gastroprotezione cronica, N13 per le statine nella dislipidemia, N66 per gli antinfiammatori non steroidei e N97 per gli anticoagulanti orali diretti nella fibrillazione atriale non valvolare) scelte perché coprono insieme l'intera varietà di condizioni che una nota può imporre: soglie su valori continui, diagnosi multiple, eccezioni e interazioni fra parametri. La verifica empirica si fonda su un insieme di riferimento di 122 casi clinici annotati manualmente (il *gold standard*, nella terminologia della letteratura), rispetto ai quali il comportamento del sistema viene confrontato con quello di due alternative naturali: un LLM lasciato a sé stesso e un LLM dotato di retrieval ma privo del componente simbolico.

Il valore scientifico del lavoro non sta tuttavia nell'architettura in sé, ma nelle domande a cui essa permette di rispondere. L'intera valutazione è organizzata attorno a sette domande di ricerca, d'ora in avanti abbreviate RQ (dall'inglese *Research Question*) e numerate da RQ1 a RQ7, che fanno da filo conduttore a tutti i capitoli che seguono; ciascuna è formulata in modo da ammettere una risposta binaria sul comportamento osservato del sistema. Le prime due riguardano i due componenti presi singolarmente: il componente simbolico decide come l'annotatore umano sui casi di riferimento (RQ1)? E il retrieval individua, per ogni caso, i passaggi delle Note che sostengono la decisione (RQ2)? Va detto subito, a scanso di equivoci, che annotatore dei casi e autore delle regole coincidono: la risposta a RQ1 misura quindi la coerenza interna fra codifica e annotazione, una verifica necessaria ma non una validazione clinica esterna, e come tale va letta in tutto ciò che segue (la questione è ripresa e discussa nel Capitolo 5). La terza e la sesta riguardano la spiegazione: il testo generato dal modello si appoggia effettivamente alle fonti recuperate, senza

introdurre informazioni esterne (RQ3)? E consente al medico di ricondurre ogni affermazione al testo della nota (RQ6)? La quarta interroga la robustezza del sistema rispetto a ripetizioni e a perturbazioni controllate dell'input (RQ4); la quinta ne misura il vantaggio rispetto alle baseline (RQ5); la settima, infine, sintetizza in un giudizio complessivo l'utilità clinica e la conformità ai requisiti di sicurezza (RQ7). Rispondere a queste sette domande, con le riserve metodologiche che ciascuna porta con sé, è l'obiettivo scientifico del lavoro: tutto ciò che le precede, dal contesto clinico all'architettura del sistema, ne costituisce la premessa necessaria, il contorno che dà sostanza alle domande senza sostituirle.

Il contributo è triplice. Sul piano architetturale, la tesi mostra una strategia di integrazione neuro-simbolica praticabile per un CDSS prescrittivo, con un confine netto fra ciò che decide e ciò che spiega. Sul piano metodologico, propone un protocollo di valutazione strutturato per domande di ricerca, in cui ogni metrica è al servizio di una sola domanda binaria. Sul piano operativo, infine, rilascia un sistema riproducibile end-to-end, dal documento ufficiale AIFA fino ai numeri finali della valutazione.

Coerentemente con questa gerarchia, la tesi procede dalle premesse alle risposte. Il Capitolo 1 ricostruisce il contesto clinico e regolatorio delle Note AIFA e descrive le quattro Note oggetto di studio; il Capitolo 2 introduce gli strumenti tecnici (rule engine, pipeline RAG, approcci neuro-simbolici) e le metriche di valutazione che torneranno nei risultati; il Capitolo 3 presenta l'architettura, motivando il principio di separazione fra decisione e spiegazione; il Capitolo 4 descrive le scelte implementative concrete, dallo stack tecnologico ai principali algoritmi del rule engine e della pipeline di ingestione. Il cuore scientifico del lavoro viene dopo: il Capitolo 5 fissa l'impianto della valutazione, ossia l'insieme dei casi di riferimento e il protocollo sperimentale, ed enuncia le sette domande di ricerca; il Capitolo 6 le affronta una per una, ciascuna in una sezione propria, con i risultati numerici e la discussione critica delle limitazioni residue; il Capitolo 7 ne sintetizza le sette risposte e indica gli sviluppi futuri. L'Appendice A raccoglie i comandi e le versioni che permettono di rieseguire l'intera valutazione da PDF a numeri canonici.

Capitolo 1

Il contesto clinico delle Note AIFA

Il Servizio Sanitario Nazionale (SSN) italiano garantisce l'accesso ai farmaci essenziali attraverso un sistema di rimborsabilità che tiene insieme sostenibilità economica e appropriatezza clinica. Per un sottoinsieme rilevante dei medicinali di fascia A, la rimborsabilità non è automatica: il prescrittore deve verificare che il paziente soddisfi i criteri definiti in atti normativi di dettaglio denominati *Note AIFA*. Questa tesi si occupa di automatizzare tale verifica per quattro Note selezionate, costruendo un sistema di supporto alla decisione clinica capace di produrre un esito motivato e tracciabile a partire dal profilo clinico del paziente e dal testo normativo ufficiale [2]. Il corpus di valutazione comprende 122 casi di riferimento, ciascuno corrispondente a uno scenario clinico realistico; la distribuzione delle etichette riflette l'eterogeneità delle situazioni prescrittive: 69 casi RIMBORSABILE, 39 casi NON_RIMBORSABILE, 9 casi NON_DETERMINABILE e 5 casi instradati verso una Nota dipendente.

1.1 Sistemi di supporto alla decisione clinica

Un sistema di supporto alla decisione clinica (CDSS, *Clinical Decision Support System*) è un software progettato per assistere il professionista sanitario nell'elaborazione di informazioni cliniche al fine di migliorare la qualità delle decisioni [3]. La definizione è intenzionalmente ampia: include sistemi di allerta per le interazioni farmacologiche, strumenti di scoring prognostico, motori di verifica normativa e raccomandatori terapeutici. La distinzione rilevante per questo lavoro è quella tra sistemi attivi, che intervengono proattivamente nel flusso di lavoro clinico, e sistemi passivi, che rispondono a una richiesta puntuale del medico [4].

La storia dei CDSS inizia nel 1974 con MYCIN [5], un sistema per la selezione della terapia antibiotica sviluppato a Stanford da Shortliffe. MYCIN codificava la conoscenza clinica esperta come insieme di regole di produzione *if-then* pesate

con fattori di certezza, e dimostrava per la prima volta che un software poteva raggiungere prestazioni paragonabili a quelle del clinico esperto su un dominio ben delimitato. Negli anni successivi, il modello si è evoluto verso linguaggi formali interoperabili: l'Arden Syntax [6], standardizzato da Health Level Seven (HL7) nel 1992, ha fornito un framework per specificare moduli di logica clinica integrabili nelle cartelle elettroniche, spostando l'enfasi dalla knowledge base monolitica alla modularità delle regole. Il decennio 2010 ha visto l'integrazione progressiva di metodi probabilistici e di apprendimento automatico nei CDSS, con guadagni sulla generalizzazione ma perdite sulla tracciabilità: un modello che impara pattern dai dati non produce per costruzione una catena di giustificazioni verificabile [7].

Il contesto delle Note AIFA pone requisiti specifici che limitano l'applicabilità dei soli modelli probabilistici. La decisione di rimborsabilità ha effetto economico e amministrativo diretto: ogni prescrizione non conforme genera oneri sul SSN e può comportare responsabilità disciplinare per il medico. Una componente rule-based deterministica è dunque indispensabile per garantire l'auditabilità del percorso decisionale. Al tempo stesso, i dati clinici reali sono raramente completi, e la Nota può contemplare parametri non rilevati al momento della prescrizione: è necessario gestire formalmente l'incertezza senza rinunciare alla tracciabilità. Il sistema descritto in questa tesi risponde a questi requisiti combinando un motore di regole simbolico con un modulo di Retrieval-Augmented Generation, il cui ruolo complementare è analizzato a partire dal Capitolo 3.

1.2 La regolamentazione AIFA della rimborsabilità

L'Agenzia Italiana del Farmaco (AIFA) è l'ente pubblico indipendente, posto sotto la vigilanza del Ministero della Salute e del Ministero dell'Economia e delle Finanze, che regola l'intero ciclo di vita del medicinale in Italia: valutazione tecnico-scientifica, negoziazione del prezzo con il produttore, classificazione in fascia di rimborsabilità e farmacovigilanza post-marketing [2]. La spesa farmaceutica pubblica si attesta intorno ai quindici miliardi di euro annui, ed è il perimetro all'interno del quale l'AIFA esercita il suo mandato di sostenibilità.

I medicinali commercializzati in Italia vengono classificati in tre grandi fasce: la fascia A comprende i farmaci essenziali e quelli per patologie croniche ad elevato impatto sociale, erogati gratuitamente salvo eventuale ticket regionale; la fascia H riguarda i medicinali riservati all'uso ospedaliero o specialistico, anch'essi a carico del SSN nelle strutture pubbliche; la fascia C include i farmaci il cui costo ricade interamente sul cittadino. Per un sottoinsieme significativo dei farmaci di fascia A, la

rimborsabilità non è incondizionata ma vincolata al rispetto dei criteri definiti nelle *Note AIFA*, atti pubblicati in Gazzetta Ufficiale e aggiornati con cadenza variabile al ritmo delle evidenze cliniche. Attualmente sono vigenti oltre cinquanta Note, numerate progressivamente, che coprono aree terapeutiche eterogenee: cardiovascolare, metabolica, neurologica, reumatologica, gastroenterologica, oncologica.

Ogni Nota è identificata da un numero progressivo e specifica, per una o più categorie farmacologiche, le indicazioni cliniche rimborsabili, le controindicazioni assolute, i percorsi terapeutici di primo e di secondo livello e i parametri di stratificazione del rischio. La complessità strutturale delle Note è molto variabile: una Nota semplice si riduce a una singola condizione booleana (il farmaco è rimborsabile se la diagnosi corrisponde a un codice ICD (*International Classification of Diseases*) specifico), mentre una Nota complessa come la N97 richiede il calcolo di uno score composito, la verifica di soglie differenziali per sesso, la presenza di una tabella di dosaggio con criteri di riduzione multipla e la gestione di controindicazioni assolute che escludono intere sotto-classi terapeutiche. Tale eterogeneità strutturale è costitutiva, non accessoria: è esattamente il motivo per cui la sola memorizzazione delle regole da parte del prescrittore è una strategia insufficiente su scala.

1.3 Le quattro Note oggetto di studio

Il sistema implementato in questa tesi codifica quattro Note AIFA (N01, N13, N66 e N97) selezionate secondo un criterio di eterogeneità clinica e di complessità logica crescente. La N01 costituisce il caso logicamente più semplice, riducibile a una congiunzione di due disgiunzioni. La N13 introduce la stratificazione del rischio cardiovascolare con soglie di colesterolo LDL (*Low-Density Lipoprotein*) variabili per classe di rischio. La N66 aggiunge liste chiuse di farmaci e controindicazioni farmaco-specifiche per singola molecola. La N97 è la più articolata dell'insieme: score composito, aritmetica d'intervallo, soglie differenziali per sesso e tabella di dosaggio con criteri di riduzione multipla [2]. Complessivamente, le quattro Note coprono i 122 casi di riferimento con distribuzione RIMB/NON_RIMB/NON_DET/ROUTED pari a 69/39/9/5 casi.

1.3.1 N01: gastroprotezione cronica

La N01 regola la rimborsabilità degli inibitori di pompa protonica (IPP) nella prevenzione delle complicanze gravi del tratto gastrointestinale superiore in pazienti in terapia cronica con farmaci ulcerogeni [8]. Gli IPP agiscono bloccando irreversibilmente la pompa H^+/K^+ -ATPasi delle cellule parietali gastriche, sopprimendo la secrezione

acida basale e stimolata; i cinque principi attivi rimborsabili ai sensi della Nota sono omeprazolo, esomeprazolo, lansoprazolo, pantoprazolo e rabeprazolo.

Il razionale clinico della Nota poggia sul rischio di ospedalizzazione per complicanze gravi (emorragia, perforazione, ostruzione pilorica), stimato tra l'1 e il 2% annuo nella popolazione generale in terapia con antinfiammatori non steroidei (FANS), e aumentato fino a quattro-cinque volte nelle categorie a rischio specificate dalla Nota stessa. La condizione necessaria di contesto è la co-somministrazione di un FANS o di acido acetilsalicilico a basse dosi; la condizione sufficiente per la rimborsabilità è la presenza di almeno uno tra i fattori di rischio elencati, fra i quali figurano pregressa emorragia digestiva, ulcera peptica non guarita, terapia anticoagulante concomitante, terapia corticosteroidica concomitante ed età avanzata (criterio asserted dal clinico, non definito numericamente nel testo normativo). La struttura logica è dunque una congiunzione tra la condizione di contesto e la disgiunzione dei fattori di rischio: la semplicità di questa topologia non elimina però la complessità operativa, poiché la N01 presenta un meccanismo di *routing* inter-Nota, in cui la rimborsabilità dell'associazione diclofenac/misoprostolo dipende dall'esito della N66, introducendo una dipendenza tra documenti normativi distinti.

1.3.2 N13: ipolipemizzanti nella dislipidemia

La N13 regola la rimborsabilità degli ipolipemizzanti nel trattamento delle dislipidemie, introducendo un principio di proporzionalità tra rischio cardiovascolare e soglia di trattamento [9]. L'ipercolesterolemia è il principale fattore di rischio cardiovascolare modificabile su scala di popolazione: una riduzione dell'LDL-C di 1 mmol/L si associa a una riduzione del rischio di eventi cardiovascolari maggiori di circa il 22% [10]. La Nota non prescrive di trattare ogni paziente iperlipemico, ma afferma che la soglia dell'LDL-C che giustifica la rimborsabilità è tanto più bassa quanto più elevato è il rischio cardiovascolare stimato: un paziente a rischio molto alto deve raggiungere un target LDL inferiore a 70 mg/dL, mentre un paziente a rischio medio può restare non trattato farmacologicamente se il colesterolo non supera 130 mg/dL.

I farmaci in scope comprendono le statine (simvastatina, pravastatina, fluvastatina, lovastatina, atorvastatina, rosuvastatina), l'ezetimibe, i fibrati e gli acidi grassi polinsaturi omega-3. La stratificazione del rischio segue le linee guida della *European Society of Cardiology* (ESC) e della *European Atherosclerosis Society* (EAS) [11] e definisce cinque classi: basso (rischio SCORE $\leq 1\%$), medio (2–3%), moderato (4–5%), alto (5–10%) e molto alto ($\geq 10\%$ o malattia cardiovascolare conclamata), con target LDL rispettivamente di sola modifica dello stile di vita, inferiore a 130, 115, 100 e 70 mg/dL. Un'eccezione trasversale riguarda i pazienti con dislipidemie familiari monogeniche:

per questi, indipendentemente dallo score calcolato, i farmaci di primo e secondo livello sono rimborsabili senza la necessità di dimostrare il mancato raggiungimento del target con la sola dieta. Questa regola di *bypass* a priorità superiore rispecchia la logica clinica secondo cui il rischio genetico assoluto giustifica l'intervento farmacologico indipendentemente dal rischio epidemiologico calcolato.

1.3.3 N66: antinfiammatori non steroidei

La N66 regola la rimborsabilità degli antinfiammatori non steroidei nella gestione del dolore e dell'infiammazione in quattro contesti clinici specifici: artropatie su base connettivitica, osteoartrosi in fase algica o infiammatoria, dolore neoplastico, attacco acuto di gotta [12]. I FANS agiscono mediante inibizione della ciclo-ossigenasi (COX) dell'acido arachidonico, bloccando la sintesi di prostaglandine e trombossani responsabili della risposta infiammatoria; la distinzione farmacologicamente rilevante è tra gli inibitori non selettivi della COX, che espongono il paziente a rischio gastrointestinale elevato, e i coxib (celecoxib, etoricoxib), inibitori selettivi della COX-2 a minor rischio gastrointestinale ma con profilo di rischio cardiovascolare aumentato.

La Nota disciplina 28 principi attivi, e la rimborsabilità non è un attributo della categoria FANS in generale ma di ciascun principio attivo singolarmente. La nimesulide è ammessa solo per il trattamento di breve durata del dolore acuto; l'associazione ibuprofene/codeina è rimborsabile quando il dolore non sia adeguatamente controllato con analgesici singoli; i coxib sono esclusi dalla rimborsabilità nei pazienti con malattia cardiovascolare conclamata o insufficienza renale grave, indipendentemente dall'indicazione. Quest'ultimo meccanismo, una controindicazione farmaco-specifica che sovrascrive l'esito positivo della verifica dell'indicazione, è la caratteristica logicamente più impegnativa della Nota e motiva l'introduzione dell'operatore $\text{IN}(\text{farmaco}, \mathcal{L}_{\text{FANS}})$ nel linguaggio del motore di regole.

1.3.4 N97: anticoagulanti orali nella fibrillazione atriale

La N97 regola la rimborsabilità di quattro anticoagulanti orali diretti (DOAC), ossia dabigatran, rivaroxaban, apixaban ed edoxaban, nella fibrillazione atriale non valvolare (FANV) [13]. La FANV è la più comune aritmia cardiaca sostenuta nella popolazione adulta: in assenza di profilassi anticoagulante, il rischio di ictus ischemico è aumentato da due a sette volte rispetto alla popolazione generale, ragione per cui l'anticoagulazione a lungo termine è indicata per la maggior parte dei pazienti con diagnosi confermata [14].

Il percorso decisionale della Nota segue quattro fasi sequenziali: conferma elettrocardiografica della FANV; quantificazione del rischio tromboembolico mediante il punteggio CHA₂DS₂-VASc; valutazione del rischio emorragico con la scala HAS-BLED, che pesa fattori quali ipertensione, alterata funzione renale o epatica, pregresso ictus o sanguinamento ed età avanzata; selezione del farmaco e del dosaggio appropriato in base alle comorbidità renali ed epatiche. Il punteggio CHA₂DS₂-VASc assegna pesi a sette condizioni cliniche: scompenso cardiaco (+1), ipertensione (+1), età ≥ 75 anni (+2), diabete mellito (+1), pregresso ictus o attacco ischemico transitorio (TIA) (+2), vasculopatia (+1) e sesso femminile (+1), per un massimo di 9 punti. La soglia di rimborsabilità dipende dal sesso: un punteggio CHA₂DS₂-VASc ≥ 2 nei maschi e ≥ 3 nelle femmine è necessario per giustificare il trattamento rimborsabile. Le controindicazioni assolute, ossia emorragia maggiore in atto, diatesi emorragica congenita, gravidanza, ipersensibilità documentata al farmaco, protesi valvolare meccanica o FANV valvolare (per i soli DOAC), producono un esito NON_RIMBORSABILE immediato, verificato prima di qualsiasi altra condizione. La N97 conta ventidue regole nel formato YAML del motore di regole ed è l'unica delle quattro Note a richiedere l'intera gamma degli operatori logici implementati, inclusa l'aritmetica d'intervallo per la gestione dei parametri clinici parzialmente noti.

1.4 Il problema della verifica della rimborsabilità

La verifica della conformità di una prescrizione a una Nota AIFA è, nella pratica corrente, affidata alla memoria e alla competenza del clinico senza alcun ausilio sistematico. L'errore prescrittivo non è un evento raro né privo di conseguenze: l'inappropriatezza di una singola prescrizione si traduce in spesa farmaceutica non dovuta per il SSN ma può anche determinare danno clinico quando una controindicazione assoluta, come l'insufficienza renale grave nei pazienti trattati con determinati DOAC, viene trascurata in favore dell'indicazione principale. La letteratura documenta come la complessità cognitiva del processo prescrittivo, amplificata dal numero e dalla densità normativa delle Note, sia associata a tassi non trascurabili di non conformità con oneri tanto economici quanto di sicurezza per il paziente [4].

La difficoltà è strutturale. Una Nota complessa come la N97 richiede al medico di tenere simultaneamente in mente la diagnosi di FANV, il calcolo di uno score a otto variabili con aritmetica intera, le soglie differenziali per sesso, una tabella di dosaggio dipendente dalla funzione renale e un insieme di controindicazioni assolute che si applicano in modo disomogeneo ai quattro principi attivi. Ogni parametro clinico non rilevato introduce un margine di incertezza che il giudizio umano gestisce

implicitamente, mentre un sistema formale richiede regole esplicite per la propagazione del valore ignoto attraverso la catena decisionale.

L'automazione di questa verifica, mediante un sistema che integri la lettura delle Note con i dati clinici del paziente e produca un esito motivato e tracciabile, è la motivazione applicativa centrale di questa tesi. La progettazione dell'architettura, la scelta del formalismo logico per la gestione dell'incertezza e il protocollo di valutazione empirica sono descritti a partire dal Capitolo 3.

È su questo terreno clinico e regolatorio che prendono forma le domande di ricerca che guidano il lavoro. L'eterogeneità logica delle quattro Note (dalla semplice congiunzione della N01 allo score composito della N97) è ciò che rende non banale la prima domanda, se cioè un componente simbolico possa replicare il giudizio dell'annotatore umano sull'intera varietà dei casi (RQ1); la posta clinica della verifica, in cui una controindicazione trascurata può tradursi in danno, è ciò che impone alle domande successive di misurare non solo l'utilità del sistema, ma la sua sicurezza, la fedeltà della spiegazione alle fonti e la tracciabilità dell'intero percorso decisionale. Il capitolo seguente introduce gli strumenti tecnici con cui queste domande verranno affrontate.

Capitolo 2

Fondamenti

Il Capitolo 1 ha mostrato perché la verifica di una Nota AIFA (Agenzia Italiana del Farmaco) è un problema clinico e regolatorio insieme, e perché un CDSS (*Clinical Decision Support System*) rule-based puro, pur essendo lo stato dell'arte nei gestionali italiani, non basta a produrre quella spiegazione tracciabile che il medico richiede di fronte a una decisione che giudica controintuitiva. Per costruire un sistema che affronti entrambi i limiti servono tre strumenti tecnici, di cui questo capitolo offre un'esposizione mirata al solo livello di dettaglio che i capitoli successivi assumeranno come acquisito. Il primo è un linguaggio per rappresentare l'incertezza clinica quando un parametro di laboratorio è assente: la logica trivalente di Kleene, discussa nella Sezione 2.1 insieme al paradigma dei sistemi a regole che la ospita. Il secondo è il meccanismo con cui ancorare le spiegazioni al testo normativo ufficiale invece che alla memoria parametrica di un modello generativo: la Sezione 2.2 descrive l'architettura RAG e le sue varianti di retrieval. Il terzo è il framework concettuale che permette di combinare i due: la Sezione 2.3 colloca il sistema nel paesaggio degli approcci neuro-simbolici e motiva la scelta dell'accoppiamento debole. Il capitolo si chiude con il vocabolario delle metriche che ricompariranno nel Capitolo 6.

2.1 Sistemi a regole e reasoning simbolico

La nozione di ragionamento automatico su base simbolica ha radici profonde nell'informatica e nella logica matematica. Un sistema a regole, detto anche *sistema di produzione* o *rule engine*, rappresenta la conoscenza come un insieme di proposizioni condizionali della forma *se condizione allora azione*, che un motore di inferenza valuta sistematicamente su un insieme di fatti [6]. La semplicità della struttura nasconde una potenza espressiva considerevole: ogni regola è leggibile da un esperto di dominio, ogni passo inferenziale è tracciabile e ogni decisione può essere motivata elencando

le regole che hanno contribuito ad attivarla. Queste proprietà rendono i sistemi a regole particolarmente adatti a contesti safety-critical, dove la trasparenza del processo decisionale non è un lusso ma un requisito normativo [3].

Il meccanismo di inferenza più comune nei sistemi a regole orientati ai fatti è il *forward chaining*: partendo dai fatti noti, il motore cerca le regole le cui condizioni siano soddisfatte, ne applica le azioni, aggiunge i nuovi fatti all'insieme di lavoro e itera fino a quando non è più possibile attivare alcuna regola. L'alternativa, il *backward chaining*, parte dall'obiettivo da dimostrare e cerca a ritroso le condizioni che lo implicano; è la strategia tipica dei sistemi Prolog-like e degli oracoli di interrogazione. Il forward chaining è più naturale per la verifica di ammissibilità normativa, dove si parte da un insieme di parametri clinici noti e si vuole determinare se soddisfano cumulativamente i criteri di una Nota AIFA: il motore applica le regole nell'ordine in cui i fatti si rendono disponibili, senza bisogno di conoscere a priori l'obiettivo.

Una proprietà del forward chaining rilevante per l'applicazione clinica è la *monotonicità* dell'inferenza rispetto all'acquisizione di nuovi dati: aggiungere un fatto all'insieme di lavoro non può rendere false le conclusioni già raggiunte a meno che non si disponga di regole di retrazione esplicita. Questa caratteristica, nota anche come *closed-world monotonicity* nel senso in cui l'insieme dei fatti cresce ma non contrae le conclusioni già validate, è importante perché garantisce che la decisione prodotta dal rule engine rimanga stabile durante la sessione clinica, finché non arrivano nuovi dati contraddittori.

2.1.1 Logica di Kleene a tre valori

La logica classica binaria postula che ogni proposizione sia vera o falsa. In un contesto clinico reale, questo assunto è quasi sempre violato: un parametro di laboratorio può essere non ancora disponibile, un referto può essere atteso, un campo della cartella clinica può non essere stato compilato. Un sistema binario che gestisce queste situazioni come *false* è strutturalmente inadeguato, perché equipara l'assenza di informazione alla confutazione. La logica trivalente di Kleene (K3) [15] introduce un terzo valore, convenzionalmente indicato come **U** (*Undefined* o *Unknown*), che denota una proposizione il cui valore di verità non è ancora determinabile. Bochvar aveva formulato indipendentemente un calcolo analogo [16], ma è la caratterizzazione algebrica di Kleene a essere adottata nei sistemi a regole moderni per la sua coerenza con le operazioni proposizionali standard.

Formalmente, sia $\mathcal{V} = \{\mathbf{T}, \mathbf{F}, \mathbf{U}\}$ l'insieme dei valori di verità. Le operazioni fondamentali si definiscono come estensione delle tavole di verità classiche con il principio di minima informazione: se almeno un operando è **U** e il risultato non è

determinabile dagli altri operandi, il risultato è **U**. Le tavole di verità per congiunzione e disgiunzione rispettano questo principio:

$$A \wedge B = \min(A, B) \quad \text{e} \quad A \vee B = \max(A, B)$$

rispetto all'ordine $\mathbf{F} < \mathbf{U} < \mathbf{T}$. La negazione si comporta classicamente su **T** e **F**, mentre $\neg \mathbf{U} = \mathbf{U}$. Alcune proprietà notevoli emergono immediatamente: $A \wedge \mathbf{F} = \mathbf{F}$ indipendentemente da A , il che significa che una condizione falsificante nota è sufficiente a determinare un risultato negativo anche in presenza di dati mancanti; dualmente, $A \vee \mathbf{T} = \mathbf{T}$ indipendentemente da A .

Oltre all'ordine di verità, K3 ammette un distinto *ordine informativo* $\sqsubseteq$ definito da $\mathbf{U} \sqsubseteq \mathbf{T}$ e $\mathbf{U} \sqsubseteq \mathbf{F}$, con **T** e **F** incomparabili tra loro. Rispetto a $\sqsubseteq$, le operazioni $\wedge$, $\vee$ e $\neg$ sono monotone: se $A \sqsubseteq A'$ e $B \sqsubseteq B'$, allora $A \wedge B \sqsubseteq A' \wedge B'$ e $A \vee B \sqsubseteq A' \vee B'$. Operativamente questo significa che acquisire nuovi dati clinici, cioè sostituire **U** con un valore determinato, raffina monotonicamente l'esito della formula complessiva, senza mai ribaltarne arbitrariamente il segno: il sistema può passare da **U** a **T** o a **F**, ma sempre in modo coerente con la struttura logica delle regole.

Nel sistema di questa tesi, il valore **U** corrisponde al campo clinico non compilato o al parametro non ancora misurato. Quando tutte le condizioni di una Nota AIFA producono **T**, il rule engine emette la decisione RIMBORSABILE; quando almeno una condizione produce **F** e nessun cammino alternativo è percorribile, emette NON_RIMBORSABILE; quando la presenza di **U** in posizione critica rende il risultato non determinabile con i dati disponibili, emette NON_DETERMINABILE. Questa tricotomia è un'istanza diretta della semantica K3 e la sua implementazione nel motore è verificata analiticamente da test di unità sulle tavole di verità.

Il quadro storico dei sistemi esperti a regole in medicina (da MYCIN [5] ad Arden Syntax [6] ai CDSS moderni [3, 4, 7]) è già stato richiamato nella Sezione 1.1. Qui basta osservare che i sistemi rule-based puri offrono tracciabilità completa e aggiornabilità normativa ma soffrono di rigidità espressiva, mentre i modelli di apprendimento automatico offrono espressività ma soffrono di opacità: nessuna delle due famiglie è da sola sufficiente per un sistema che debba essere allo stesso tempo corretto per costruzione sul piano normativo e capace di produrre una spiegazione in linguaggio naturale auditabile. È la motivazione che porta, nella Sezione 2.3, alla scelta neuro-simbolica.

2.2 Recupero e generazione condizionata

I grandi modelli linguistici (*Large Language Model*, LLM) sono addestrati su corpora testuali di dimensioni enormi e acquisiscono conoscenza del mondo in forma distribuita nei loro parametri. Questa *conoscenza parametrica* è però statica: il modello non può aggiornare autonomamente ciò che sa senza un nuovo ciclo di addestramento, e non ha accesso diretto a documenti che non ha visto durante il training. In un dominio a conoscenza chiusa come le Note AIFA, dove i criteri di rimborsabilità cambiano per decreto ministeriale e devono essere applicati nella loro formulazione letterale, fare affidamento esclusivo sulla memoria parametrica del modello produce due categorie di errori: l'allucinazione di criteri inesistenti e l'omissione di criteri aggiornati dopo la data di cutoff [17]. La *Retrieval-Augmented Generation* (RAG) è stata proposta da Lewis et al. come soluzione a questo problema: invece di rispondere esclusivamente da memoria, il modello riceve come contesto i passaggi documentali recuperati da un corpus esterno che il sistema può aggiornare indipendentemente dall'addestramento [18].

2.2.1 Architettura di un sistema RAG

Una pipeline RAG canonica si articola in due fasi temporalmente separate. La fase *offline*, eseguita una tantum o ad ogni aggiornamento del corpus, costruisce un indice dei documenti: il testo viene segmentato in unità, ciascuna unità viene convertita in un vettore numerico tramite un modello di embedding, e i vettori vengono memorizzati in un database specializzato per la ricerca per similarità. La fase *online*, eseguita ad ogni interrogazione, converte la domanda dello stesso modello di embedding in un vettore query, recupera i vettori di documento più simili, e fornisce al modello generativo tanto la domanda originale quanto i passaggi recuperati come contesto esplicito. Il modello produce così una risposta che è condizionata sui documenti, non soltanto sulla propria memoria parametrica [19].

Un sistema RAG combina quindi un componente di recupero, che seleziona dai documenti i passaggi rilevanti per una domanda, con un modello generativo, che produce una risposta condizionata su quei passaggi. Quando i documenti hanno valore normativo (come accade per le Note AIFA, dove ogni frase è vincolante) questa architettura riduce gli errori di allucinazione e, soprattutto, fornisce una catena di citazioni che il revisore umano può controllare in tempo lineare rispetto al numero di fonti citate.

2.2.2 Chunking e indicizzazione

Il *chunking*, la segmentazione del testo in unità di dimensione adatta all'embedding, è una delle scelte progettuali più critiche in una pipeline RAG, perché condiziona sia la qualità del retrieval sia la lunghezza del contesto consegnato al modello generativo. Unità troppo grandi contengono più informazione ma diluiscono il segnale semantico e rischiano di superare la finestra di contesto del modello; unità troppo piccole sono più precise ma possono separare passaggi che il modello deve leggere insieme per costruire una risposta coerente.

Le strategie di chunking si suddividono grossolanamente in due famiglie. Il *fixed-size chunking* divide il testo ogni w token, eventualmente con una finestra di sovrapposizione di ampiezza $s < w$ per preservare la continuità semantica ai confini: produce chunk di dimensione prevedibile e si implementa in modo semplice, ma ignora la struttura del documento. Il *semantic chunking* invece sfrutta la struttura del documento (paragrafi, sezioni, elenchi) per produrre unità semanticamente coerenti; richiede un analizzatore sintattico o una conoscenza della struttura del documento, ma produce chunk che corrispondono a unità informative discrete, particolarmente utili con testi normativi dove ogni paragrafo di solito esprime un criterio autonomo.

L'indicizzazione dei chunk avviene in un *database vettoriale*, una struttura dati specializzata per la ricerca per similarità in spazi ad alta dimensione. I database vettoriali moderni implementano algoritmi di indicizzazione approssimata come HNSW (*Hierarchical Navigable Small World*) [1], che costruiscono un grafo navigabile a più livelli gerarchici e permettono di trovare i vicini approssimati di un vettore query in tempo logaritmico rispetto alla dimensione del corpus, a fronte di un costo di costruzione dell'indice proporzionale al numero di chunk.

2.2.3 Embedding, similarità densa e reranking

Il modello di embedding è il componente che trasforma testo in vettori. I modelli basati su trasformatori a doppio encoder (*bi-encoder*) producono una rappresentazione densa per ogni sequenza di input calcolando l'embedding da una singola passata attraverso il modello [20]. L'architettura originale del trasformatore, introdotta da Vaswani et al. con il meccanismo di *self-attention* [20], ha reso possibile la costruzione di encoder contestuali di alta qualità: il vettore di embedding di un token non è fisso ma dipende da tutti gli altri token della sequenza, permettendo di catturare relazioni semantiche a lunga distanza. BERT [21] ha poi mostrato come il pre-addestramento su grandi corpora tramite predizione mascherata produca rappresentazioni trasferibili

che, con un minimo di fine-tuning, raggiungono prestazioni all'avanguardia su decine di task di elaborazione del linguaggio naturale (*Natural Language Processing*, NLP).

Sentence-BERT (SBERT) [22] ha risolto il problema specifico della similarità semantica fra frasi: BERT originale produce embedding per token, non per sequenze, e la media o l'aggregazione dei token embedding produce rappresentazioni di scarsa qualità per il confronto semantico inter-frase. SBERT addestra reti siamesi con pooling sul token [CLS] o con mean-pooling sull'output dell'encoder, ottimizzando esplicitamente la similarità coseno fra coppie di frasi simili e dissimili. Il modello `paraphrase-multilingual-mpnet-base-v2`, adottato in questo lavoro per l'indicizzazione dei chunk, è la variante multilingue di SBERT addestrata su 50 lingue, che garantisce la coerenza delle rappresentazioni fra il testo normativo italiano e le domande cliniche degli utenti.

La similarità fra un vettore query $\mathbf{q}$ e un vettore chunk $\mathbf{d}$ si misura con la *cosine similarity*:

$$\cos(\mathbf{q}, \mathbf{d}) = \frac{\mathbf{q} \cdot \mathbf{d}}{\|\mathbf{q}\| \|\mathbf{d}\|}$$

che varia tra -1 e 1 ed è invariante per riscalamento dei vettori. Quando i vettori sono già normalizzati a norma unitaria, come accade nell'implementazione standard di SBERT e dei database vettoriali, la cosine similarity si riduce al prodotto scalare $\mathbf{q} \cdot \mathbf{d}$, che può essere calcolato efficientemente con operazioni BLAS e permette la ricerca del nearest neighbor approssimato mediante strutture HNSW.

Il retrieval denso tramite bi-encoder è rapido ma approssimato: il bi-encoder codifica query e documento indipendentemente, senza modellare l'interazione fra i due. Un approccio alternativo, storicamente dominante prima dell'era degli embedding neurali, è il *retrieval lessicale* basato sulla frequenza dei termini. Il modello BM25 (Best Match 25, nella tradizione del probabilistic relevance framework di Robertson e Zaragoza) assegna ad ogni documento un punteggio che tiene conto della frequenza del termine nella query all'interno del documento (*term frequency*, TF) e della rarità del termine nel corpus (*inverse document frequency*, IDF), con una funzione di saturazione che attenua l'effetto dei documenti molto lunghi. Il punteggio BM25 per un documento d rispetto a una query con termine t è:

$$\text{BM25}(d, t) = \text{IDF}(t) \cdot \frac{\text{TF}(t, d) \cdot (k_1 + 1)}{\text{TF}(t, d) + k_1 \cdot \left(1 - b + b \cdot \frac{|d|}{\bar{d}}\right)}$$

dove k_1 e b sono iperparametri che controllano rispettivamente la saturazione della frequenza e la normalizzazione per lunghezza, e $\bar{d}$ è la lunghezza media dei documenti nel corpus. BM25 eccelle quando la query contiene termini tecnici rari e specifici,

come i codici ATC (*Anatomical Therapeutic Chemical*) o i nomi di molecole nelle Note AIFA, ma fatica con le parafrasi semantiche, dove il documento risponde alla domanda ma usa vocabolario diverso. Il retrieval denso tramite embedding ha il comportamento opposto: cattura la vicinanza semantica ma può perdere l'ancoraggio lessicale esatto.

Questa complementarità ha portato allo sviluppo di pipeline di retrieval ibride che combinano il punteggio BM25 e il punteggio coseno, e di un secondo stadio di ordinamento a latenza maggiore ma precisione più alta: il *cross-encoder reranker* [23]. Mentre il bi-encoder encode query e documento separatamente, il cross-encoder li concatena come singolo input al modello, permettendo al meccanismo di self-attention di modellare direttamente le interazioni fra i token della query e quelli del chunk. Il risultato è uno score di rilevanza più preciso, a costo di $O(k)$ passate forward aggiuntive per i k candidati da riordinare. La pipeline *two-stage* (bi-encoder a bassa latenza per il recupero di un sovrainsieme di K candidati, seguito da cross-encoder per la selezione finale dei top- k) è diventata l'architettura standard per i sistemi RAG ad alta precisione [19]. In questo lavoro, il cross-encoder **nickprock/cross-encoder-italian-bert-stsb** è addestrato in italiano su coppie di similarità semantica e opera come secondo stadio di selezione sui chunk preliminarmente recuperati dal database vettoriale.

2.3 Approcci neuro-simbolici

Un sistema *neuro-simbolico* integra componenti di apprendimento automatico (tipicamente reti neurali profonde) con componenti di ragionamento simbolico (tipicamente regole, ontologie o logiche formali). La motivazione è duplice: i sistemi puramente simbolici sono precisi e auditabili ma fragili di fronte a dati rumorosi o incompleti e incapaci di gestire input non strutturati come il testo libero; i sistemi puramente neurali generalizzano bene e gestiscono l'incertezza, ma producono ragionamenti opachi e sono inclini ad allucinazioni su fatti specifici di dominio. Un sistema neuro-simbolico mira a ereditare i pregi di entrambi, e la sua realizzazione concreta dipende dal tipo e dal grado di accoppiamento fra le due componenti [24].

La letteratura distingue tradizionalmente fra accoppiamento debole (*loose coupling*) e accoppiamento stretto (*tight coupling*). Nell'accoppiamento debole, i due sistemi interagiscono tramite interfacce ben definite: il modulo simbolico riceve l'output del modulo neurale (ad esempio un'entità estratta da testo) e vi applica regole; oppure il modulo neurale riceve l'output del modulo simbolico (ad esempio un fact verificato) e lo usa come contesto aggiuntivo. I due moduli sono addestrabili e verificabili in isolamento. Nell'accoppiamento stretto, la componente neurale e quella simbolica

sono interconnesse in modo end-to-end, spesso con gradienti che fluiscono attraverso entrambe: è il caso di DeepProbLog, dove le probabilità di regole logiche sono parametri appresi dalla rete neurale [25], oppure di sistemi dove i vincoli simbolici sono incorporati come strati differenziabili.

Nell'ambito clinico, Yu et al. argomentano che l'accoppiamento debole è spesso preferibile perché preserva la verificabilità indipendente dei due moduli [24]: un auditor può ispezionare le regole simboliche senza dover comprendere i pesi della rete neurale, e viceversa. Questa proprietà è particolarmente preziosa in contesti regolamentati come la rimborsabilità farmaceutica, dove il medico prescrittore deve poter giustificare la decisione di fronte a un controllo normativo senza fare riferimento a meccanismi opachi. Il sistema descritto in questa tesi adotta un accoppiamento debole con separazione architetturale esplicita: la componente simbolica, il rule engine che decide la rimborsabilità, non riceve mai output dalla componente neurale né viene influenzata da essa; la componente neurale, la pipeline RAG che produce la spiegazione in linguaggio naturale, riceve invece i risultati della componente simbolica come contesto strutturato da citare e amplificare.

La separazione non è soltanto una scelta di ingegneria del software: è una garanzia di correttezza formale. La decisione di rimborsabilità è una funzione deterministica dei dati clinici del paziente e del testo della Nota AIFA; non dipende dal modello linguistico, dalla sua versione, dal suo sampling o dal suo stato interno. Questo è esattamente ciò che differenzia il sistema sia dai CDSS rule-based puri, che non producono spiegazione in linguaggio naturale, sia dai sistemi RAG clinici, che non garantiscono la correttezza normativa della decisione. I modelli basati su grandi modelli linguistici come Med-PaLM [26] o i sistemi LLM per la medicina in generale [27] ottengono risultati impressionanti su benchmark clinici, ma la loro correttezza rimane probabilistica e non auditabile riga per riga; non sono adatti a sostituire un sistema normativo dove ogni errore ha conseguenze legali e sanitarie.

La separazione fra decisione deterministica ed esplicazione probabilistica è pertanto non una limitazione del sistema ma il suo principale vantaggio architetturale: essa rende la componente di decisione formalmente verificabile a prescindere dalla qualità della generazione testuale, e rende la componente di generazione migliorabile o sostituibile senza toccare la logica normativa.

2.4 Valutazione di sistemi RAG e CDSS

Valutare un sistema RAG in un contesto clinico non è riducibile alla misura di un'unica cifra. Il sistema deve essere corretto nella decisione, preciso nel retrieval, fedele alle

fonti nella generazione e utile all'utente nel complesso. Questi quattro aspetti sono parzialmente indipendenti: un sistema può avere un retrieval perfetto e un LLM allucinante, oppure una generazione fluente che cita fonti inesistenti, oppure, come nel sistema di questa tesi, una decisione formalmente corretta e una spiegazione di qualità variabile. Le metriche che seguono, e che sono utilizzate nella valutazione quantitativa del Capitolo 6, devono essere intese non come giudici assoluti ma come sonde su dimensioni distinte del comportamento del sistema.

2.4.1 Metriche di retrieval

La *Recall@k* misura la frazione di documenti rilevanti effettivamente restituiti fra i k risultati:

$$\text{Recall@}k = \frac{|D_k \cap \text{rel}(q)|}{|\text{rel}(q)|}$$

dove D_k è l'insieme dei chunk recuperati e $\text{rel}(q)$ l'insieme dei chunk di riferimento per la query q . Con k grande la recall tende a crescere ma la precisione decade, perché il retriever include sempre più chunk non pertinenti che occupano la finestra di contesto del modello. La *Precision@k* complementa la recall misurando la frazione di chunk recuperati che sono effettivamente rilevanti:

$$\text{Precision@}k = \frac{|D_k \cap \text{rel}(q)|}{k}$$

Nel sistema di questa tesi, la *Precision@3* e la *Precision@5* sono particolarmente alte (0,989 e 0,970 rispettivamente), il che indica che quasi tutti i chunk recuperati sono pertinenti. Questo risultato è spiegabile con la struttura delle Note AIFA, dove ogni chunk corrisponde a un criterio autonomo e i criteri non pertinenti sono raramente recuperati perché condividono poco vocabolario con la query del caso in esame.

Il *Mean Reciprocal Rank* (MRR) valuta la posizione del primo documento rilevante nella lista ordinata, mediata su tutte le query:

$$\text{MRR} = \frac{1}{|Q|} \sum_{q \in Q} \frac{1}{\text{rank}_1(q)}$$

dove $\text{rank}_1(q)$ è la posizione del primo chunk rilevante per q . Un MRR pari a 1,0 indica che il chunk rilevante è sempre in prima posizione: è il risultato osservato nel sistema di questa tesi, coerente con la struttura di retrieval two-stage dove il cross-encoder affina l'ordinamento del bi-encoder. Il *Normalized Discounted Cumulative Gain* (NDCG@k) generalizza MRR pesando la posizione di tutti i documenti rilevanti con una funzione logaritmica decrescente, ma risulta meno informativo in questo contesto perché il

corpus ha una struttura binaria (un chunk è rilevante o non lo è) e la Recall@k ne cattura già il comportamento saliente.

2.4.2 Metriche di fedeltà

La *fedeltà* (*faithfulness*) di una risposta misura in quale misura le affermazioni contenute nella risposta sono supportate dai chunk recuperati, indipendentemente dalla loro correttezza assoluta rispetto alla realtà. Una risposta può essere fattualmente corretta ma non fedele ai chunk (se il modello attinge alla propria memoria parametrica) o fedele ai chunk ma fattualmente errata (se i chunk stessi contengono un errore). In un contesto normativo, la fedeltà è la proprietà più critica, perché garantisce che la spiegazione prodotta dal sistema sia ancorata ai documenti ufficiali e non a inferenze autonome del modello.

Il framework RAGAS [28] implementa la fedeltà come stima della frazione di affermazioni nella risposta entailed dai chunk di contesto: il modello LLM-giudice decompone prima la risposta in proposizioni atomiche, poi verifica per ciascuna se è logicamente implicata dal contesto recuperato, e infine calcola la proporzione delle proposizioni supportate. Questo approccio è semantico ma dipende dalla qualità dell'LLM-giudice e risente dell'ambiguità nelle definizioni di entailment, motivo per cui la versione *strict*, che richiede entailment letterale anziché semantico, è computata separatamente come limite superiore di rigore.

Un approccio complementare si basa sull'inferenza del linguaggio naturale (*Natural Language Inference*, NLI): un modello NLI multi-label (nel sistema di questa tesi il modello mDeBERTa-v3-base-xnli-multilingual-nli-2mil7) classifica ogni frase della risposta come entailed, neutral o contraddittoria rispetto alla concatenazione dei chunk di contesto, e la fedeltà NLI è la proporzione di frasi entailed. L'NLI ha il vantaggio di essere locale e non dipendente dall'LLM-giudice, ma è più rigida della fedeltà semantica e tende a sottostimare la fedeltà quando la risposta parafrasa il testo normativo invece di citarlo letteralmente.

2.4.3 Qualità della risposta, compositi e statistica

La fedeltà valuta la relazione fra risposta e contesto, ma non misura se la risposta sia utile per chi ha posto la domanda. La *Answer Relevancy* di RAGAS [28] misura invece la coerenza fra la risposta e la domanda originale: il modello genera n domande sintetiche a partire dalla risposta prodotta, le codifica come vettori di embedding, e calcola la similarità coseno media fra i vettori delle domande sintetiche e il vettore della domanda originale. Un punteggio alto indica che la risposta contiene l'informazione

necessaria a rispondere alla domanda; uno basso segnala che la risposta è fuori tema o generica.

QAEval [29] offre un approccio alternativo alla valutazione della qualità basato su micro-domande: a partire dalla risposta di riferimento (*gold answer*), viene generato un insieme di domande di verifica elementari, ciascuna delle quali sonda un fatto specifico della risposta attesa. La risposta candidata è poi valutata controllando quante di queste micro-domande ricevono la risposta corretta, producendo una misura di copertura fattuale fine-grained. Il QAEval F1 combina precision e recall sulle micro-domande analogamente a una misura di information extraction. Nel sistema di questa tesi il QAEval F1 medio è 0,172, un valore basso che riflette la difficoltà di coprire tutti i dettagli normativi della risposta di riferimento con un modello locale di 8 miliardi di parametri quantizzato a 4 bit (Llama 3.1 8B Q4_K_M).

Le metriche descritte finora sono eterogenee per scala, natura e peso diagnostico. Per produrre una valutazione globale del sistema occorre aggregarle in modo esplicito e giustificato. Il composito *ALES* (*Aggregated LLM Evaluation Score*) adottato in questa tesi è una media pesata di quattro componenti:

$$ALES = 0,40 \cdot D + 0,30 \cdot E + 0,20 \cdot U + 0,10 \cdot Q$$

dove D è il *DecisionScore* (correttezza della decisione normativa), E è l'*EvidenceSupport* (fedeltà alle fonti, combinazione pesata con pesi efficaci 0,435 per la copertura *verbatim* a 3-grammi (il componente dominante), 0,174 per l'entailment NLI e 0,261 per la RAGAS faithfulness; la variante *strict* non è prodotta dal backend RAGAS impiegato e non contribuisce al punteggio), U è la *ContextualUtility* (media pesata 0,75 della *context precision* e 0,25 della *context recall*, poiché la *context relevancy* non è restituita dal backend RAGAS adottato) e Q è la *AnswerQuality* (combinazione 0,70 di QAEval F1 e 0,30 di Answer Relevancy). Il peso dominante assegnato a D (0,40) riflette la gerarchia di priorità del sistema: la correttezza della decisione normativa è il requisito non negoziabile, e gli altri componenti ne misurano la qualità della spiegazione. I valori dei pesi sono impostati come costanti nel modulo di valutazione e non vengono ottimizzati sui dati di test, per evitare di adattare la misura ai risultati.

Poiché molte metriche producono proporzioni o medie su campioni di dimensione finita, la loro incertezza statistica deve essere quantificata per non confondere la variabilità campionaria con differenze sistemiche. Il metodo del *bootstrap percentile* [30] ricampiona con rimpiazzo l'insieme dei n casi per $B = 5000$ iterazioni e calcola l'intervallo di confidenza al 95% come il 2,5° e il 97,5° percentile della distribuzione dei campioni bootstrap. Per le proporzioni binarie (come il *pass rate* del rule engine) è disponibile anche l'intervallo esatto di Wilson [31], che per proporzioni vicine a 0 o

1 è superiore al bootstrap percentile perché non richiede approssimazione normale. L'intervallo di Wilson per una proporzione osservata $\hat{p}$ su n osservazioni con livello di confidenza corrispondente a z è:

$$\frac{\hat{p} + \frac{z^2}{2n} \pm z\sqrt{\frac{\hat{p}(1-\hat{p})}{n} + \frac{z^2}{4n^2}}}{1 + \frac{z^2}{n}}$$

Con $\hat{p} = 1,000$ e $n = 122$ casi, l'intervallo di Wilson al 95% è $[0,970; 1,000]$: il rule engine ottiene una classificazione senza errori sul corpus di riferimento, con un limite inferiore conservativo del 97%.

I tre pilastri concettuali (reasoning simbolico con logica trivalente, pipeline RAG con retrieval two-stage, integrazione neuro-simbolica a basso accoppiamento) non sono stati scelti per completezza enciclopedica ma perché ciascuno risponde a un requisito preciso del sistema descritto nel Capitolo 3, dove i loro contorni concettuali diventano contorni architetturali e i loro vincoli formali diventano vincoli di sicurezza clinica.

È anche il vocabolario di queste pagine a rendere formulabili le domande di ricerca. La logica trivalente fissa il significato delle tre decisioni che RQ1 mette alla prova; le metriche di retrieval (Recall@ k , MRR) sono le sonde di RQ2; le metriche di fedeltà e il composito ALES traducono in numeri le domande sulla spiegazione (RQ3 e RQ6) e sul giudizio complessivo (RQ7); gli intervalli di confidenza bootstrap e di Wilson danno alle risposte la cautela statistica che ciascuna richiede. Gli strumenti, insomma, non valgono per sé, ma per le domande che permettono di porre in modo misurabile.

Capitolo 3

Architettura del sistema

Il Capitolo 2 ha definito tre strumenti: un linguaggio per rappresentare l'incertezza, una pipeline per ancorare le risposte ai documenti, un confine concettuale fra ragionamento simbolico e generativo. La domanda di progetto è come comporli in un sistema unico che rimanga clinicamente sicuro, cioè in cui i fallimenti possibili del componente generativo, che è intrinsecamente non verificabile per costruzione [17, 27], non possano mai ribaltare una decisione di rimborsabilità. La risposta che questo capitolo sviluppa è la *separazione neuro-simbolica delle responsabilità*: la correttezza normativa è interamente affidata a un rule engine deterministico ispezionabile, mentre il modello generativo riceve la decisione già calcolata e ha l'unico compito di articolarne la motivazione in linguaggio clinico ancorato alle Note. La discussione procede dai requisiti che il sistema deve soddisfare, attraverso il principio architetturale che li governa, fino alla descrizione analitica di ciascun componente e del contratto che li lega.

3.1 Requisiti e separazione neuro-simbolica

Il sistema viene progettato per operare su quattro Note AIFA (Agenzia Italiana del Farmaco): la N01 (inibitori di pompa protonica), la N13 (ipolipemizzanti), la N66 (antinfiammatori non steroidei) e la N97 (anticoagulanti orali nella fibrillazione atriale non valvolare). Il corpus di valutazione comprende 122 casi di riferimento raccolti per coprire tutti i percorsi decisionali ammissibili.

Sul piano funzionale, il sistema deve soddisfare sette obiettivi distinti, ciascuno con un criterio di accettazione misurabile. Il primo è la verifica formale della rimborsabilità: il sistema deve produrre un esito appartenente all'insieme

$$\mathcal{D} = \{\text{RIMBORSABILE}, \text{NON_RIMBORSABILE}, \text{NON_DETERMINABILE}, \text{ROUTED}\}$$

con Macro F1 pari a 1,0 sull'intero corpus di riferimento. Il secondo è la spiegazione in linguaggio naturale: ogni risposta deve contenere almeno un'evidenza che citi esplicitamente il PDF sorgente, il numero di pagina e un estratto verbatim dalla norma applicata. Il terzo è la citazione di estratti normativi verificabili: la copertura delle citazioni deve essere del 100 %, nel senso che ogni decisione deve associare almeno un'ancora *file + page + excerpt* al testo PDF. Il quarto requisito riguarda la gestione dell'incertezza: l'assenza di un campo clinico nel profilo paziente si deve propagare come **UNKNOWN** nella logica a tre valori, senza imporre valori di default né degradare silenziosamente verso **FALSE**, il che trasformerebbe un caso indeterminato in un rifiuto non fondato. Il quinto requisito è la modalità what-if: deve essere possibile modificare un campo del profilo paziente e rieseguire la sola valutazione simbolica senza reinvocare il layer linguistico, qualora la decisione non sia cambiata. Il sesto è la segnalazione strutturata dei campi mancanti: in caso di esito **NON_DETERMINABILE**, la risposta deve riportare la lista esatta delle variabili cliniche la cui assenza ha causato l'indeterminazione, in modo che il clinico sappia quali informazioni raccogliere per risolvere l'ambiguità. Il settimo e ultimo requisito funzionale è la tracciabilità e l'audit della catena decisionale: l'oggetto di risposta deve includere la sequenza ordinata delle regole valutate, con l'identificatore, il valore Kleene e l'ancora normativa per ciascuna, esportabile ai fini di verifica regolatoria.

Sul piano non funzionale, quattro proprietà condizionano l'intera architettura. Il *determinismo* delle decisioni cliniche richiede che, a parità di input, l'esito simbolico sia sempre identico, indipendentemente dall'ordine di valutazione o dallo stato interno dei processi; questo esclude qualsiasi fonte di stocasticità nel layer decisionale. La *verificabilità per-caso* impone che ogni singola decisione possa essere rieseguita e auditata in isolamento, senza bisogno di avviare l'intero sistema: il rule engine simbolico deve essere esercitabile autonomamente sull'insieme di riferimento anche quando il layer RAG (*Retrieval-Augmented Generation*) non è attivo. La *riproducibilità canonica* della valutazione richiede che le metriche siano misurabili su un ambiente riproducibile da zero: wipe del database vettoriale, re-ingestione da PDF puliti e valutazione end-to-end, con dipendenze Python fissate tramite hash crittografici. Infine, la *latenza accettabile per uso clinico interattivo* non impone requisiti sub-secondo (l'interazione con un medico prescrivente tollera qualche secondo di elaborazione), ma esclude tempi di risposta dell'ordine dei minuti che renderebbero il sistema inutilizzabile nella pratica quotidiana. Un vincolo trasversale di progetto è il deployment interamente locale: nessun dato clinico deve lasciare la macchina su cui il sistema è installato, in coerenza con i requisiti di riservatezza dei dati sanitari e con le limitazioni infrastrutturali degli ambienti ospedalieri italiani.

La separazione fra decisione e spiegazione non è soltanto una scelta architetturale:

è un vincolo di sicurezza. Un large language model, per quanto accurato nelle sue previsioni medie, non offre garanzie formali sulla coerenza fra il suo output e una specifica normativa scritta [17]. L'allucinazione, la produzione di affermazioni plausibili ma non fondate, è una proprietà intrinseca dei modelli autoregressivi, non un difetto che la selezione del modello o l'aumento della scala risolvono completamente [27]. Un rule engine, al contrario, è verificabile per costruzione: ogni decisione è la conseguenza deterministica di una catena di regole esplicite e ispezionabili. Per questo motivo, nel sistema proposto, l'unica responsabilità del modello generativo è la produzione del testo esplicativo, mentre la determinazione della rimborsabilità è interamente affidata al layer simbolico.

Il trade-off è reale e va riconosciuto onestamente. Il layer simbolico richiede che le regole normative siano trascritte manualmente in un formato strutturato, un lavoro che introduce la possibilità di errori di codifica non presenti nel testo PDF originale; il sistema di test e i casi di riferimento mitigano questo rischio ma non lo eliminano del tutto. Il layer linguistico introduce latenza aggiuntiva, stocasticità residua anche con temperatura fissata a zero, e la necessità di un controllo post-generazione che verifichi la coerenza fra il testo prodotto e la decisione deterministica già calcolata. Entrambi i layer, inoltre, hanno footprint computazionale e devono coesistere su hardware commodity. La scelta di separarli architetturealmente trasforma questi rischi in dipendenze gestibili: il rule engine può essere testato esaustivamente in isolamento, e il layer linguistico può essere sostituito o aggiornato senza toccare la logica decisionale.

Definizione 3.1 (Architettura neuro-simbolica con separazione delle responsabilità). Il sistema $\mathcal{S}$ è detto *neuro-simbolico con separazione delle responsabilità* se e solo se esiste una partizione $\mathcal{S} = \mathcal{S}_{\text{sym}} \cup \mathcal{S}_{\text{neu}}$ tale che: (i) $\mathcal{S}_{\text{sym}}$ è verificabile formalmente su un test set $\mathcal{T}$ senza invocare alcun componente di $\mathcal{S}_{\text{neu}}$; (ii) $\mathcal{S}_{\text{neu}}$ riceve come input l'output già calcolato di $\mathcal{S}_{\text{sym}}$ e non può modificarlo; (iii) il contratto fra i due strati è espresso da un tipo strutturato con schema validato a runtime.

Il secondo principio architettureale che governa il design è il *fail-fast*: ogni fase di validazione deve fallire immediatamente e in modo esplicito al primo invariante violato. Nel rule engine questo si traduce in fasi ordinate con uscite di short-circuit esplicite; nel layer RAG si traduce nel controllo post-generazione che rileva le contraddizioni fra la decisione deterministica e il testo prodotto dall'LLM prima che raggiungano il clinico. Uno stato corrotto silenzioso è più pericoloso di un errore esplicito in un sistema che supporta decisioni cliniche.

I principi di *single source of truth* e di *traceability* completano il quadro: le regole normative hanno un'unica rappresentazione (i file YAML versionati), e ogni decisione

deve essere ricondotta a una catena esplicita e riproducibile di passi, ognuno con l'ancora nel PDF che ne è la fonte autorizzativa.

3.2 Architettura complessiva

Il sistema è articolato in sei componenti principali, organizzati in tre livelli funzionali: il livello decisionale simbolico, il livello esplicativo neurale e il livello di presentazione. La Figura 3.1 ne illustra la *deployment view*, mostrando i confini di processo, i protocolli di comunicazione e il flusso di dati sia nella fase online di valutazione sia nella fase offline di ingestione dei PDF.

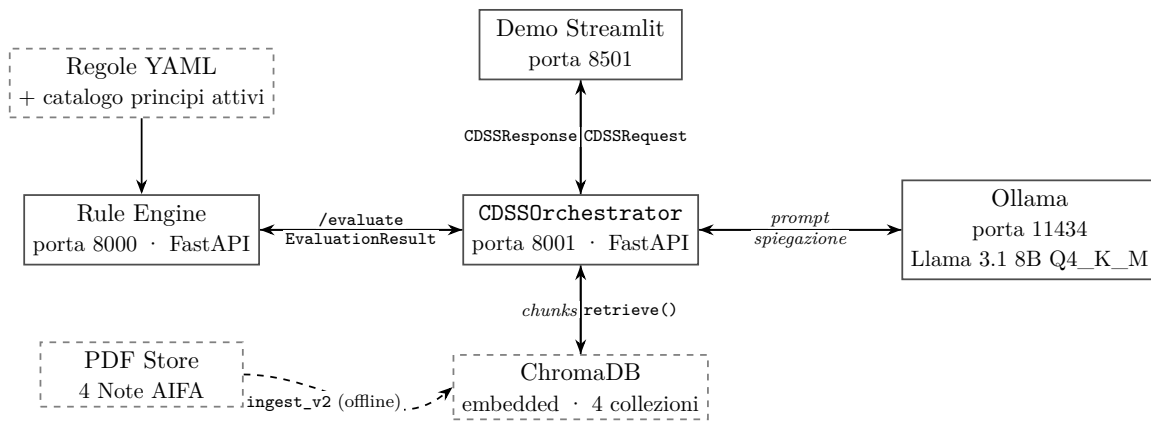


Figura 3.1: Deployment view dell'architettura a componenti. Le frecce continue rappresentano il flusso online durante la valutazione (REST/HTTP tra processi separati; filesystem per ChromaDB e PDF); la freccia tratteggiata indica la fase offline di ingestione: i PDF normativi vengono estratti, segmentati e indicizzati nelle quattro collezioni ChromaDB [1].

Il Rule Engine risiede in un microservizio autonomo esposto sulla porta 8000 tramite FastAPI [32] e Pydantic [33]. La scelta di isolarlo come processo separato risponde direttamente al requisito di verificabilità per-caso: l'intero insieme di riferimento di 122 casi può essere esercitato senza che Ollama sia in esecuzione, riducendo drasticamente il tempo di ogni ciclo test-fix-verify. Il costo di questa scelta è un'interfaccia REST intermedia che introduce una latenza aggiuntiva nell'ordine dei millisecondi, trade-off accettabile dato che l'interazione clinica non impone requisiti sub-secondo.

L'Orchestratore, in esecuzione sulla porta 8001, costituisce il componente di integrazione: interroga il Rule Engine tramite HTTP/REST [34], raccoglie i chunk pertinenti da ChromaDB e compone il prompt strutturato per Ollama [35]. Il canale REST fra Rule Engine e Orchestratore è formalmente tipizzato tramite Pydantic: ogni scambio di dati attraversa uno schema validato, eliminando la categoria di errori dovuta a contratti impliciti.

ChromaDB opera in modalità *embedded* con persistenza su filesystem locale, eliminando i rischi di disponibilità connessi al teorema CAP (*Consistency, Availability, Partition tolerance*) [36] che affliggono le soluzioni distribuite. Il deployment è interamente locale: nessun dato clinico lascia la macchina su cui il sistema è installato, in coerenza con i requisiti di riservatezza dei dati sanitari e con le limitazioni infrastrutturali degli ambienti ospedalieri italiani. La containerizzazione Docker, coordinata da un `docker-compose.yml` che fissa le versioni di tutti i servizi, garantisce la riproducibilità dell'ambiente di esecuzione indipendentemente dall'hardware host.

La demo interattiva Streamlit (porta 8501) fornisce un'interfaccia grafica per la valutazione manuale dei casi: riceve il profilo paziente, invoca l'Orchestratore e presenta la decisione simbolica insieme alla spiegazione testuale con le evidenze citate.

3.3 Il layer simbolico: rule engine

Il Rule Engine implementa la componente simbolica del sistema. La sua responsabilità esclusiva è la determinazione dell'esito di rimborsabilità: dato un profilo paziente e una prescrizione, restituisce un oggetto `EvaluationResult` strutturato che incapsula la decisione, la traccia di copertura per ogni regola valutata, i flag clinici derivati e la lista dei campi mancanti che hanno causato eventuali indeterminazioni.

3.3.1 Modello dei dati e schema YAML

Il profilo paziente è rappresentato dal tipo `PatientProfile`, che aggrega tre sotto-profilo: dati anagrafici (età, sesso, peso, altezza), profilo clinico (diagnosi attive, comorbidità, terapie in corso) e profilo cardiovascolare (componenti dello score CHA₂DS₂-VASc, funzionalità renale ed epatica [37]). Tutti i campi sono opzionali per costruzione: l'assenza di un valore è rappresentata come `None` Python e si propaga come `UNKNOWN` nella logica di Kleene a tre valori (Sezione 2.1). La scelta di rendere tutti i campi opzionali, anziché definire un sottoinsieme obbligatorio, risponde a un'esigenza del dominio: nella pratica clinica reale il profilo paziente è spesso incompleto al momento della prescrizione, e un sistema che rigettasse le richieste incomplete risulterebbe inutilizzabile proprio nei casi più frequenti e più bisognosi di supporto. Il campo `missing_fields` di `EvaluationResult` costituisce il canale strutturato attraverso cui il sistema comunica al clinico esattamente quali informazioni aggiuntive permetterebbero di risolvere un esito `NON_DETERMINABILE`.

Le regole normative sono codificate in un Domain-Specific Language (DSL) in formato YAML [38], che trasforma ogni regola in un dato strutturato: un file di testo versionabile, leggibile da personale clinico non tecnico e aggiornabile al ritmo

degli aggiornamenti normativi senza che l'interprete debba essere ricompilato. Ogni regola include obbligatoriamente cinque campi: un identificatore univoco (`rule_id`), il tipo di regola nell'insieme `{SCOPE, PATH, DOSE, EXCEPTION, EXCLUSION}`, un campo `evaluation_order` che determina la posizione nella pipeline, un'ancora normativa con file PDF, pagina e estratto verbatim, e un albero di condizioni espresso tramite operatori logici componibili (`AND`, `OR`, `NOT`, `IS_TRUE`, `SCORE_RANGE_GTE` e altri). Si è preferito YAML rispetto a JSON per la leggibilità della sintassi e rispetto a un linguaggio di regole proprietario per la disponibilità di tooling standard; il costo è l'assenza di tipizzazione statica, compensata dalla validazione Pydantic all'avvio del servizio.

3.3.2 Tre famiglie di regole e logica Kleene

Le regole del DSL si articolano in tre famiglie logiche distinte, corrispondenti ai nodi decisionali delle Note AIFA. Le *regole di inclusione* (tipo `PATH` e `INCLUSION`) verificano che il paziente soddisfi almeno uno dei criteri positivi previsti dalla Nota: se nessun criterio di inclusione risulta soddisfatto, l'esito è `NON_RIMBORSABILE` con short-circuit immediato. Le *regole di esclusione* (tipo `EXCLUSION`) verificano l'assenza di controindicazioni assolute: se almeno una esclusione vale `TRUE`, l'esito è `NON_RIMBORSABILE` con priorità sulle inclusioni. Le *regole di eccezione* (tipo `EXCEPTION`) gestiscono i casi di routing verso un'altra Nota o di bypass anticipato della pipeline: il loro esito non è né rimborsabilità né non-rimborsabilità, ma `ROUTED`, che indica al clinico di rivolgersi al percorso prescrittivo di una Nota diversa.

Definizione 3.2 (Processo decisionale del Rule Engine). Sia $\mathcal{R}$ l'insieme ordinato delle regole applicabili a una coppia (`nota_id`, `drug_id`), e sia $v_K: \text{Regola} \times \text{PatientProfile} \rightarrow \{\text{TRUE}, \text{FALSE}, \text{UNKNOWN}\}$ la funzione di valutazione Kleene. Il processo decisionale del Rule Engine è la funzione $\delta: \mathcal{R} \times \text{PatientProfile} \rightarrow \mathcal{D}$, con $\mathcal{D}$ l'insieme dei quattro esiti introdotto nella Sezione 3.1, definita dalla pipeline a undici fasi (numerate 0–10): la fase 0 valida l'input e computa le variabili derivate; le fasi 1–5 corrispondono ai nodi `SCOPE`, `EXCEPTION`, `EXCL_HARD`, `INCLUSION` e `PATHWAY`, ciascuna con uscita di short-circuit immediata; le fasi 6–8 accumulano flag clinici; la fase 9 risolve gli eventuali conflitti fra flag di guidance; la fase 10 applica l'invariante di sicurezza e produce l'esito finale. Il risultato è `NON_DETERMINABILE` se e solo se nessuna regola ha prodotto un short-circuit e almeno una condizione ha propagato `UNKNOWN`.

La logica di Kleene a tre valori [15] permea l'intero processo di valutazione: ogni operatore del DSL è implementato in aritmetica Kleene, garantendo che l'incertezza si propaghi correttamente attraverso le composizioni logiche. In particolare, l'operatore

SCORE_RANGE_GTE opera in aritmetica Kleene sui punteggi parziali: se uno o più componenti dello score sono UNKNOWN, il confronto con la soglia non degrada a FALSE ma rimane UNKNOWN, cosicché il sistema segnali l'indeterminazione anziché negare erroneamente la rimborsabilità.

3.4 Il layer esplicativo: pipeline RAG

La componente neurale del sistema è una pipeline RAG progettata per fornire all'LLM una finestra di contesto strettamente ancorata al testo PDF delle Note AIFA [18]. La progettazione affronta quattro problemi specifici del dominio normativo italiano: la natura tabellare e a clausole annidate dei PDF; il vocabolario ibrido (terminologia latina farmacologica, acronimi inglesi, perifrasi cliniche italiane); la necessità di recuperare chunk che mantengano l'unità sintattica delle clausole booleane composte; e il rischio di allucinazione nelle citazioni normative.

3.4.1 Ingestione e chunking

La fase di ingestione, denominata `ingest_v2`, è una fase offline che si esegue una tantum su ogni aggiornamento dei PDF. L'estrazione del testo grezzo è realizzata tramite PyMuPDF [39], che preserva la struttura a pagina dei documenti e consente di associare ogni chunk al numero di pagina sorgente, informazione indispensabile per le ancore normative. Il chunker adottato è *sentence-aware*: il target è di 1800 caratteri per chunk, con un overlap di due frasi fra chunk consecutivi. La scelta del target a 1800 caratteri è motivata dalla finestra di 512 token del modello di embedding: a una densità media di circa 4 caratteri per token sull'italiano normativo, 1800 caratteri sono codificabili senza troncamento. L'overlap a livello di frase, anziché a livello di caratteri, preserva l'unità sintattica delle clausole normative, in particolare le condizioni booleane composte ("in pazienti con...oppure..."), che un overlap fisso a caratteri spezzerebbe a metà. Un manifest di ingestione registra il checksum SHA-256 di ogni PDF e di ogni chunk indicizzato, garantendo la tracciabilità della base documentale.

3.4.2 Modello di embedding, indice e retriever

Il modello di embedding è `paraphrase-multilingual-mpnet-base-v2`, a 768 dimensioni. La scelta del multilinguismo è motivata dal vocabolario ibrido dei PDF AIFA, che alternano terminologia latina farmacologica, acronimi clinici (CHA₂DS₂-VASc, NAO per nuovi anticoagulanti orali, VFG per velocità di filtrazione glomerulare)

e perifrasi cliniche in italiano. Un modello monolingue italiano non gestirebbe la sottocomponente latina; un modello generalista inglese degraderebbe la qualità sulle perifrasi normative italiane [22]. I vettori sono indicizzati in ChromaDB con metrica coseno, organizzati in quattro collezioni distinte (una per Nota), così da concentrare il retrieval sulla Nota pertinente alla richiesta in esame.

Il retriever applica una strategia in due passi: recupero top- k per similarità coseno, seguito da un reranker cross-encoder `nickprock/cross-encoder-italian-bert-stsb` che riordina i candidati per rilevanza contestuale [23]. Il reranker è stato scelto specificamente per il dominio italiano, in quanto è addestrato su coppie di frasi in italiano, e consente di distinguere chunk tematicamente vicini ma normalmente irrilevanti (ad esempio, tabelle di dosaggio) da chunk che contengono la clausola condizionale pertinente al caso in esame.

3.4.3 Generazione e vincolo di fedeltà

Il modello generativo è Llama 3.1 8B Q4_K_M, eseguito localmente tramite Ollama [35] con parametri fissi (`temperature=0`, `seed=42`) per garantire la riproducibilità della generazione. Il prompt è strutturato in sezioni rigide: la *decisione del Rule Engine* viene iniettata prima del contesto normativo, in una sezione marcata `[DECISIONE DETERMINISTICA -- NON CONTRADDIRE]`; il *contesto normativo* contiene i chunk recuperati, ognuno preceduto da un’etichetta `[FONTE: pdf, p. n]`; una sezione *FONTI* in coda al prompt è ricomposta deterministicamente dal sistema, non dall’LLM.

Definizione 3.3 (Insieme dei chunk rilevanti). Sia $\mathcal{C}$ l’insieme di tutti i chunk indicizzati per una Nota. Dato un profilo paziente p e un `EvaluationResult` e , l’insieme dei chunk rilevanti $\mathcal{C}^*(p, e) \subseteq \mathcal{C}$ è l’output del retriever a due stadi: la prima fase seleziona i k_1 chunk con similarità coseno massima rispetto alla query $q(p, e)$; la seconda fase applica il reranker `nickprock/cross-encoder-italian-bert-stsb` e seleziona i $k_2 \leq k_1$ chunk con score cross-encoder più alto, dove k_2 è un parametro del sistema fissato a progetto. Il testo della spiegazione finale deve citare almeno un chunk in $\mathcal{C}^*(p, e)$ e non può contraddire la decisione contenuta in e .

Un controllo post-generazione, l’*invariante di sicurezza*, confronta la decisione testuale prodotta dall’LLM con la decisione deterministica del Rule Engine: in caso di divergenza, la spiegazione viene scartata e sostituita da un template conservativo che riporta la decisione corretta e invita il clinico a consultare direttamente il testo normativo. La Figura 3.2 illustra il flusso complessivo dalla richiesta al testo di risposta.

3.5 Contratto API e flusso end-to-end

L'Orchestratore è il componente di integrazione del sistema. Esso espone verso l'esterno due endpoint: `/explain` e `/health`. L'endpoint `/explain` riceve una `CDSSRequest` in formato JSON, invoca il Rule Engine tramite `/evaluate` e, condizionatamente all'esito simbolico, attiva la pipeline RAG per la generazione della spiegazione. L'endpoint `/health` consente al sistema di monitoraggio e al processo di ingestione di verificare la raggiungibilità del servizio prima di avviare operazioni critiche.

Il contratto API è versionato tramite i campi `schema_version` (3.3) e `engine_version` (3.4.0), esposti in ogni risposta. La relazione fra i due campi è deliberata: `schema_version` identifica il contratto JSON degli oggetti scambiati, mentre `engine_version` identifica la versione del codice che interpreta le regole YAML; un aggiornamento normativo che non modifica il contratto JSON incrementa soltanto `engine_version`, mentre un cambiamento nello schema degli oggetti Pydantic richiede l'aggiornamento di entrambi. Questo disaccoppiamento consente ai client (la demo, gli script di valutazione) di gestire la compatibilità in modo esplicito.

La gestione dei casi `NON_DETERMINABILE` merita attenzione separata. Quando il Rule Engine restituisce questo esito, l'Orchestratore non sopprime la richiesta alla pipeline RAG: genera comunque una spiegazione, ma etichettata esplicitamente come *informativa*, distinta dalla spiegazione di una decisione definitiva. Il testo informa il clinico su quali campi mancanti impediscono la determinazione e quali valori ipotetici, se confermati, porterebbero a un esito positivo o negativo; questa modalità è la base della funzionalità *what-if*, il quinto dei requisiti funzionali sopra elencati.

I casi `ROUTED` vengono invece gestiti con un pattern di rinvio: l'Orchestratore restituisce nella risposta l'identificatore della Nota verso cui il caso è stato instradato, lasciando al client il compito di reindirizzare la richiesta al percorso prescrittivo corretto. Il testo esplicativo accompagna il routing con le motivazioni normative (ad esempio: il farmaco prescritto non rientra nell'ambito della N01 ma potrebbe rientrare nella N13 per indicazioni ipolipemizzanti).

Sul piano della robustezza, l'Orchestratore implementa un meccanismo di timeout esplicito per le chiamate a Ollama: se il modello non risponde entro il limite configurato, viene restituito il template conservativo dell'invariante di sicurezza, che almeno garantisce al clinico la decisione simbolica corretta anche in assenza della spiegazione testuale.

L'architettura sviluppata in questo capitolo si regge su un solo principio: la separazione netta fra il layer decisionale simbolico, formalmente verificabile e deterministico, e il layer esplicativo neurale, utile e ricco ma intrinsecamente stocastico. Il rule

engine garantisce la correttezza normativa attraverso una pipeline a fasi ordinate con logica Kleene, regole YAML versionabili e ancore normative verificabili; la pipeline RAG garantisce la qualità della spiegazione attraverso un chunker sentence-aware, un retriever a due stadi e un controllo post-generazione che protegge il clinico da eventuali allucinazioni del modello. Il contratto fra i due layer è espresso da tipi Pydantic validati a runtime, e il sistema è integralmente deployato su hardware locale per rispettare i vincoli di privacy dei dati clinici. Questa separazione è anche ciò che rende nette le domande di ricerca: poiché la decisione non dipende mai dal modello generativo, RQ1 può chiedere se il solo layer simbolico decida come l’annotatore umano, e RQ3 e RQ6 possono interrogare la fedeltà e la verificabilità della spiegazione senza che un loro eventuale fallimento comprometta la correttezza dell’esito. Resta da mostrare come queste decisioni di progetto si traducono in codice eseguibile e riproducibile, dalla funzione `evaluate` ai parametri di generazione del modello linguistico: è il compito del Capitolo 4.

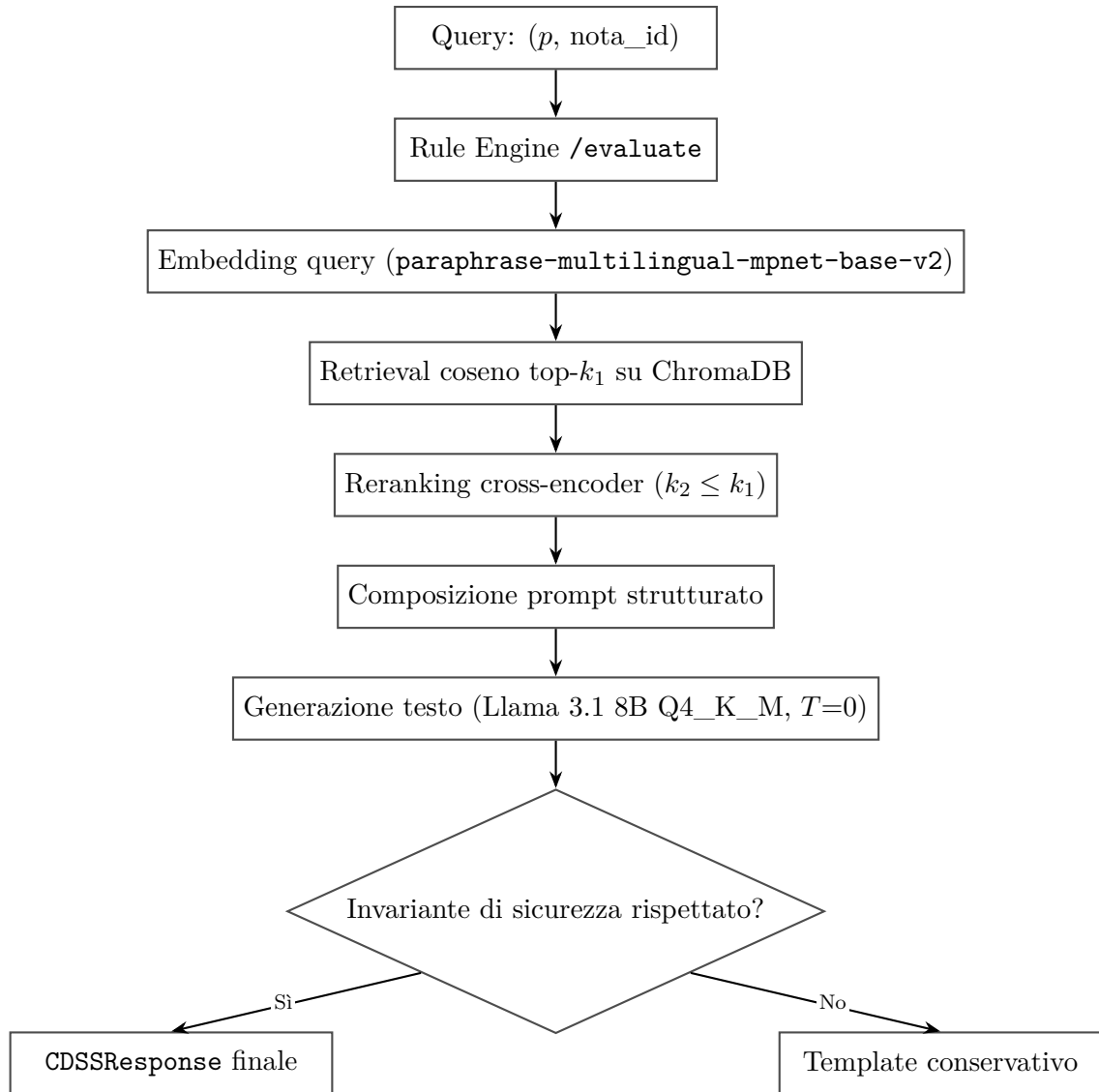


Figura 3.2: Flusso della pipeline RAG dalla richiesta al testo di risposta. Il Rule Engine viene invocato per primo e la sua decisione è iniettata nel prompt prima del contesto documentale. Il controllo dell'invariante di sicurezza in uscita dalla generazione garantisce che la spiegazione testuale non contraddica mai la decisione deterministica.

Capitolo 4

Implementazione

L'architettura del Capitolo 3 è una scelta di responsabilità: cosa decide il layer simbolico, cosa spiega il layer neurale, come si contrattano via REST. Questo capitolo mostra come quelle responsabilità si traducono in codice eseguibile, riproducibile e testato, e quali compromessi tecnologici si sono dovuti accettare per rispettare i vincoli di determinismo, riproducibilità e deployment locale fissati a progetto. Le scelte concrete (Python 3.12, FastAPI, ChromaDB embedded, Ollama, Pydantic) non sono frutto di una rassegna comparativa esaustiva: sono le opzioni che la combinazione di richiesta clinica e ambiente di calcolo disponibile restringeva a un'unica soluzione naturale. La trattazione segue l'ordine logico dei componenti, dallo stack tecnologico alla demo interattiva, ricalcando la mappa concettuale del capitolo precedente per mantenerne la leggibilità.

4.1 Stack tecnologico e organizzazione del codice

L'intero sistema è scritto in Python 3.12, scelto per la maturità dell'ecosistema scientifico, la tipizzazione statica progressiva introdotta dalle versioni recenti e la facilità di integrazione con le librerie di machine learning necessarie alla pipeline RAG (*Retrieval-Augmented Generation*). Il codice ammonta a circa 16 000 righe non vuote e non di commento, esclusi i test, distribuite fra quattro moduli principali contenuti nella directory `Note_AIFA/` del repository.

Il modulo `aifa_rule_engine/` implementa tutta la logica simbolica del motore: circa 3 500 righe tra regole YAML, valutatore, logica trivalente e API FastAPI [32] esposta sulla porta 8000. Il modulo `rag_pipeline/` comprende l'ingestione, il retriever ChromaDB, il costruttore di prompt, i backend LLM (*Large Language Model*) e l'orchestratore FastAPI sulla porta 8001, per circa 2 700 righe. Il modulo `evaluation/` è il più esteso (circa 8 800 righe) e raccoglie i runner di metriche, le baseline, i test di

robustezza e gli script overnight: una sproporzione che riflette la scelta di fare della valutazione, più che dell'implementazione, il cuore del lavoro. Il modulo `demo/`, infine, realizza l'interfaccia Streamlit in circa 1 100 righe.

Il modello linguistico locale è Llama 3.1 8B Q4_K_M, servito tramite Ollama [35] in un container dedicato; gli embedding semantici sono generati da `paraphrase-multilingual-mpnet-base-v2` [22] e il reranking utilizza `nickprock/cross-encoder-italian-bert-stsb` [40]. La persistenza semantica è affidata a ChromaDB [1], mentre la validazione degli schemi JSON è delegata a Pydantic [33].

I tre servizi (rule engine, orchestratore e Ollama) sono orchestrati da Docker Compose su una rete interna isolata dal traffico esterno. La comunicazione avviene su socket TCP locali; l'orchestratore può caricare il rule engine direttamente in-process (variabile d'ambiente `AIFA_RULES_DIR`) per eliminare la latenza HTTP nei test di integrazione. La suite di test conta 1025 casi verdi: 435 per il rule engine e 590 per l'orchestratore RAG.

4.2 Rule engine: implementazione

Il rule engine deterministico è alla versione 3.4.0 ed è scritto in Python puro, senza dipendenze da framework di reasoning esterni, con l'obiettivo di mantenere ogni passo computazionale ispezionabile riga per riga. Il catalogo delle regole è codificato in file YAML per ciascuna nota (`rules/nota_01/`, `nota_13/`, `nota_66/`, `nota_97/`) e caricato al momento dell'avvio del servizio dal modulo `engine/rule_loader.py`. Ogni documento YAML è validato contro uno schema Pydantic che garantisce la presenza dei campi obbligatori prima che il motore entri in produzione.

4.2.1 Schema YAML delle regole

Ogni voce del catalogo specifica un identificatore univoco, il tipo logico della regola, la versione di schema, la descrizione in italiano, l'ordine di valutazione, un'*anchor* normativa che punta alla pagina e alla sezione del PDF AIFA (Agenzia Italiana del Farmaco) di riferimento, e la condizione booleana strutturata. Il seguente estratto è tratto da `rules/nota_13/rules.yaml` e illustra una regola di inclusione e una di eccezione:

```
- rule_id: N13_INCL_001
  rule_type: INCLUSION
  nota: '13'
  version: 3.3.0
```

```
description_it: Dieta appropriata seguita per almeno 3 mesi.
evaluation_order: 10
normative_anchor:
  pdf_file: nota-13.pdf
  page: 1
  section: Condizioni di prescrizione
  excerpt: "dieta, seguita per almeno tre mesi"
condition:
  operator: IS_TRUE
  var: dieta_seguita_almeno_3_mesi

- rule_id: N13_EXCEPT_001
  rule_type: EXCEPTION
  nota: '13'
  version: 3.3.0
  outcome_if_true: BYPASS
  bypasses:
    - N13_INCL_001
  condition:
    operator: AND
    operands:
      - {operator: IS_TRUE, var: rischio_molto_alto}
      - {operator: GTE, var: ldl, value: 70}
```

Il campo `normative_anchor` consente al motore di tracciare ogni decisione fino alla pagina esatta del documento normativo, soddisfacendo il requisito di spiegabilità citazionale discusso nel Capitolo 3.

4.2.2 Tipi di regole e fasi di valutazione

Il valutatore implementa undici fasi sequenziali, numerate da 0 a 10 in coerenza con la Definizione 3.2. La fase iniziale valida l'input e computa le variabili derivate; la fase SCOPE verifica l'applicabilità della nota al caso clinico; la fase EXCEPTION identifica eventuali bypass delle regole di inclusione prima di valutarle; la fase EXCL_HARD applica le esclusioni assolute con fail-fast su `TRUE`; la fase INCLUSION valuta i prerequisiti del trattamento con fail-fast su `FALSE`; la fase PATHWAY verifica il percorso terapeutico; le fasi GUIDANCE_DOSE, GUIDANCE_PREF e GUIDANCE_WARN producono i flag clinici accessori senza influire sulla decisione binaria, e un'apposita

fase ne risolve gli eventuali conflitti. La fase di finalizzazione aggrega le evidenze, applica l'invariante di sicurezza e determina l'esito.

4.2.3 Implementazione della logica Kleene

La semantica trivalente è stata richiamata nella Sezione 2.1.1; qui se ne descrive soltanto la codifica concreta. Il modulo `logic/three_valued.py` esporta l'enum `TruthValue` con i valori `TRUE`, `FALSE` e `UNKNOWN`, e implementa le tavole di verità esplicitamente mediante confronti con `None`: ogni comparazione che coinvolge un campo non valorizzato restituisce `UNKNOWN`. La logica non è delegata a un solutore di terze parti, garantendo il pieno controllo sul comportamento in presenza di dati incompleti e la copertura analitica per test di unità.

L'operatore esteso `SCORE_RANGE_GTE` implementa l'*interval arithmetic* per i punteggi clinici compositi come il `CHA2DS2-VASc`: i componenti ignoti contribuiscono con il minimo pari a 0 e il massimo pari al loro peso massimo, e il confronto con la soglia k restituisce `TRUE` se il punteggio minimo supera già k , `FALSE` se il massimo non la raggiunge, `UNKNOWN` negli altri casi [41].

La decisione `NON_DETERMINABILE` viene emessa quando almeno un campo in `missing_fields_coverage` è determinante per l'esito e il motore non può concludere né `RIMBORSABILE` né `NON_RIMBORSABILE`. La lista dei campi mancanti accompagna sempre la risposta, orientando il clinico verso la raccolta del dato necessario.

4.2.4 Algoritmo di valutazione

Algoritmo 1 riporta lo pseudocodice della funzione principale `evaluate(caso, indice_regole)`.

Il routing fra le quattro Note avviene prima dell'ingresso nel ciclo: il campo `note_id` della richiesta seleziona l'oggetto `RuleIndex` corrispondente, evitando qualsiasi ambiguità fra i cataloghi. Se il campo è assente o non riconosciuto, l'orchestratore restituisce un errore 422 prima di invocare il motore.

4.3 Pipeline RAG: ingestione e recupero

4.3.1 Ingestione del corpus normativo

Il modulo `rag_pipeline/ingest_v2.py` indicizza i PDF delle quattro Note AIFA in collezioni ChromaDB dedicate. La versione 2 dell'ingestione introduce il tracciamento

Algoritmo 1 Valutazione di un caso clinico (*evaluate*)

Input: *caso*: dizionario campi clinici; *indice*: RuleIndex per la nota**Output:** EvaluationResult con decisione, audit, flag clinici

```
1: vars  $\leftarrow$  COMPUTADERIVEDVARIABLES(caso)
2: bypass_set  $\leftarrow$   $\emptyset$ 
3:    $\triangleright$  Fase EXCEPTION: identifica bypass prima di EXCL e INCLUSION
4: per ogni r  $\in$  indice.exception_rules in ordine fai
5:   tv  $\leftarrow$  EVALCONDIZIONE(r.condition, vars)
6:   SCRIVIAUDIT(r, tv)
7:   se tv = TRUE allora
8:     bypass_set  $\leftarrow$  bypass_set  $\cup$  r.bypasses
9:   fine se
10: fine per
11:    $\triangleright$  Fasi SCOPE, EXCL_HARD, INCLUSION, PATHWAY: pattern comune
12: per ogni fase  $\in$  [SCOPE, EXCLHARD, INCLUSION, PATHWAY] fai
13:   pending_unknown  $\leftarrow$   $\emptyset$ 
14:   per ogni r  $\in$  indice.fase.rules in ordine fai
15:     se r.id  $\in$  bypass_set allora
16:       continua
17:     fine se
18:     tv  $\leftarrow$  EVALCONDIZIONE(r.condition, vars)
19:     SCRIVIAUDIT(r, tv)
20:     se tv = fase.fail_tv allora
21:       restituisce FINALIZZA(NON_RIMBORSABILE, audit)
22:     fine se
23:     se tv = UNKNOWN allora
24:       pending_unknown  $\leftarrow$  pending_unknown  $\cup$  {r}
25:     fine se
26:   fine per
27:   se pending_unknown  $\neq$   $\emptyset$  allora
28:     restituisce FINALIZZA(NON_DETERMINABILE, audit)
29:   fine se
30: fine per
31:    $\triangleright$  Fasi di guidance: producono flag, non modificano la decisione
32: per ogni r  $\in$  indice.guidance_rules fai
33:   tv  $\leftarrow$  EVALCONDIZIONE(r.condition, vars)
34:   se tv = TRUE allora
35:     AGGIUNGIFLAGCLINICO(r)
36:   fine se
37: fine per
38: restituisce FINALIZZA(RIMBORSABILE, audit)
```

degli offset di carattere all'interno di ogni pagina, le bounding box per chunk, il checksum SHA-256 del testo e il riconoscimento delle tabelle tramite `pdfplumber`.

L'estrazione del testo è eseguita con `PyMuPDF` per i blocchi testuali e con `pdfplumber` per le tabelle, che vengono marcate con il flag `is_in_table=True` e suddivise in un chunk per riga. Il testo puro è tokenizzato in frasi con `spaCy` (modello italiano `it_core_news_sm`); i chunk sono costruiti con una finestra scorrevole di dimensione target `TARGET_CHARS = 1800` caratteri e sovrapposizione di `OVERLAP_SENTS = 2` frasi, bilanciando la densità informativa con la granularità necessaria alle ancore citazionali [19]. I chunk sono inviati a `ChromaDB` in lotti da 100 elementi con i metadati di pagina, nota, offset e hash, usando il modello di embedding `paraphrase-multilingual-mpnet-base-v2`.

Una nota sull'esclusione deliberata: il file `nota-97-all-1.pdf` è escluso dall'indicizzazione a causa di un problema di encoding Wingdings che produce mojibake non recuperabile con le librerie disponibili. Il contenuto normativo di N97 è comunque coperto dai file `nota-97-all-2.pdf` e `nota-97-all-3.pdf`, privi del difetto.

Algoritmo 2 riporta lo pseudocodice della funzione di indicizzazione per una singola nota.

4.3.2 Recupero e reranking

La fase di recupero invoca `ChromaDB` con una query di embedding della domanda clinica e recupera i k chunk più vicini per distanza coseno. I chunk restituiti vengono poi riordinati da un cross-encoder, `nickprock/cross-encoder-italian-bert-stsb`, che calcola la rilevanza contestuale fra la query e ciascun testo in modo più accurato rispetto alla similarità vettoriale [23]. Solo i chunk che superano la soglia di rilevanza del reranker vengono inclusi nel contesto del prompt LLM, limitando il rischio di allucinazioni indotte da passaggi normativi irrilevanti [17].

4.4 Orchestratore, riproducibilità e demo

L'orchestratore è un servizio FastAPI esposto sulla porta 8001 che coordina sequenzialmente il rule engine, il retriever semantico e il modello linguistico. Il suo ciclo di vita è gestito dal context manager asincrono `lifespan`, che al momento dell'avvio inizializza il `CDSSOrchestrator` tentando prima il caricamento diretto del rule engine in-process e ricorrendo alla comunicazione HTTP solo in caso di fallimento, per eliminare la latenza di rete nei deployment integrati.

Algoritmo 2 Indicizzazione di una nota AIFA (`indicizza_nota`)

Input: `pdf_path`: percorso del PDF; `nota_id`: identificatore nota**Output:** Numero di chunk indicizzati in ChromaDB

```
1:  $hash\_pdf \leftarrow \text{SHA256}(pdf\_path)$ 
2:  $pagine \leftarrow \text{ESTRAIPAGINE}(pdf\_path)$   $\triangleright$  PyMuPDF rawdict + pdfplumber
3:  $chunk\_list \leftarrow []$ 
4: per ogni  $p \in pagine$  fai
5:    $blocchi \leftarrow \text{ESTRAIBLOCKITESTO}(p)$ 
6:    $tabelle \leftarrow \text{ESTRAITABELLE}(p)$   $\triangleright$  pdfplumber
7:   per ogni  $b \in blocchi$  fai
8:     se  $|b.text| < \text{MIN\_BLOCK\_CHARS}$  allora
9:       salta
10:    fine se
11:     $frasi \leftarrow \text{TOKENIZZAFRASI}(b.text)$   $\triangleright$  spaCy it
12:     $finestra \leftarrow []$ ;  $char\_count \leftarrow 0$ 
13:    per ogni  $s \in frasi$  fai
14:       $finestra.append(s)$ ;  $char\_count += |s|$ 
15:      se  $char\_count \geq \text{TARGET\_CHARS}$  allora
16:         $chunk\_list.append(\text{BUILDCHUNK}(finestra, p, nota\_id))$ 
17:         $finestra \leftarrow finestra[-\text{OVERLAP\_SENTS} :]$ 
18:         $char\_count \leftarrow \sum_{s \in finestra} |s|$ 
19:      fine se
20:    fine per
21:    se  $finestra \neq []$  allora
22:       $chunk\_list.append(\text{BUILDCHUNK}(finestra, p, nota\_id))$ 
23:    fine se
24:  fine per
25:   $\triangleright$  Tabelle: un chunk per riga
26:  per ogni  $riga \in \text{APPIATTISCI TABELLE}(tabelle)$  fai
27:     $chunk\_list.append(\text{BUILDCHUNKTABELLA}(riga, p, nota\_id))$ 
28:  fine per
29: fine per
30:  $\triangleright$  Upsert in batch da CHROMA_BATCH=100
31:  $coll \leftarrow \text{GETORCREATECOLLECTION}("nota\_"+ nota\_id + "\_v2")$ 
32: per ogni  $batch \in \text{PARTITION}(chunk\_list, 100)$  fai
33:    $coll.UPSERT(batch)$ 
34: fine per
35: restituisce  $|chunk\_list|$ 
```

4.4.1 Endpoint pubblici

Il servizio espone due endpoint. L'endpoint `GET /health` restituisce lo stato di inizializzazione dell'orchestratore e viene usato dai healthcheck di Docker Compose per coordinare l'ordine di avvio dei container. L'endpoint principale è `POST /explain`, autenticato tramite l'header `X-API-Key`: la chiave attesa è letta dalla variabile d'ambiente `AIFA_API_KEY` e la sua assenza rende il servizio aperto, facilitando lo sviluppo locale senza configurazione aggiuntiva.

La struttura di input del `POST /explain` comprende il campo `schema_version` (valore atteso 3.3), l'identificatore della nota (`note_id`: uno fra "01", "13", "66", "97"), il farmaco (`drug_id`) e due dizionari distinti: `patient_data` per i dati clinici misurabili e `clinician_asserted` per le asserzioni che richiedono giudizio clinico diretto e non sono automaticamente derivabili da valori numerici. La separazione dei due dizionari è una scelta progettuale deliberata: rende esplicito al chiamante quali informazioni provengono da sistemi automatici e quali richiedono l'intervento del medico.

La risposta `CDSSResponse` include la decisione di rimborsabilità (`reimbursement_decision`), la lista di flag clinici (`clinical_flags`), i chunk normativi recuperati (`retrieved_chunks`), la spiegazione testuale generata dall'LLM (`explanation`) e le citazioni delle ancore PDF (`citations`). Il motore non delega mai la decisione al modello linguistico: l'LLM produce esclusivamente la spiegazione in linguaggio naturale a partire da una decisione già fissata dal rule engine, in accordo con l'architettura neuro-simbolica descritta nel Capitolo 3.

La gestione degli errori è progettata per non esporre i dati del paziente nei messaggi di risposta HTTP: eventuali eccezioni sono tracciate server-side nel log, mentre il client riceve soltanto un messaggio generico con codice 500, prevenendo la fuga accidentale di informazioni cliniche sensibili.

4.4.2 Riproducibilità e cleanroom

La riproducibilità è garantita da tre meccanismi ortogonali. Il primo è il file `requirements.lock` con hash esatti di tutte le dipendenze Python, che impedisce aggiornamenti impliciti alle librerie. Il secondo è il manifest `ingestion_manifest_v2.json`, che registra il checksum SHA-256 di ogni PDF normativo ed è confrontato ad ogni avvio del servizio con i file presenti su disco. Il terzo è la fissazione dei parametri di generazione del modello linguistico a `temperature=0` e `seed=42`, che rende deterministica la risposta testuale a parità di input e contesto. La pipeline `run_full_overnight.sh` esegue la valutazione completa con checkpoint per

stage (ingestione, rule engine, risposte RAG, RAGAS, QAEval) e può essere ripresa dal punto di interruzione in caso di riavvio. La riproducibilità profonda è garantita dallo script `full_cleanroom_v2.sh`, che cancella le collezioni ChromaDB, reingredisce i PDF da zero, esegue la suite di test e valuta: solo questa procedura, la *cleanroom run*, esclude che i risultati siano condizionati da artefatti di esecuzioni precedenti residui nel database vettoriale, perché la valutazione sulla sola ChromaDB esistente può mascherare bug della fase di ingestione che non emergono fino al successivo wipe.

4.4.3 Suite di test e demo interattiva

La suite conta 1025 test verdi, organizzati in tre livelli. I test unitari verificano il comportamento delle singole funzioni della logica trivalente, degli operatori di confronto e del parser YAML. I test di integrazione eseguono scenari clinici completi sul rule engine (435 casi, di cui 133 specifici per le quattro Note AIFA) e sull'orchestratore RAG (590 casi). I test *property-based* scritti con Hypothesis verificano invarianti come la simmetria delle tavole di Kleene, la monotonia dell'esito al crescere dell'informazione disponibile e la preservazione delle ancore normative in tutte le risposte non vuote. La copertura del codice misurata dalla suite è del 93% sul rule engine e dell'89% sull'orchestratore.

La demo interattiva, infine, è realizzata in Streamlit sulla porta 8501. Streamlit è stata scelta per la possibilità di produrre un'interfaccia clinicamente leggibile in poche centinaia di righe di Python puro: la minore flessibilità di layout rispetto a un'applicazione React è accettabile per un prototipo destinato alla revisione clinica e non a un deployment prescrittivo. La sidebar espone in posizione fissa il disclaimer medico-legale obbligatorio; il client incapsula le chiamate al rule engine e all'orchestratore in un singolo modulo con gestione esplicita degli errori di connessione e degradazione graceful in caso di indisponibilità di uno dei servizi. La pagina di dettaglio include un pannello *what-if* che consente di modificare i parametri clinici e rieseguire la pipeline completa (rule engine e generazione della spiegazione) aggiornando interattivamente sia la decisione sia la motivazione; le identità dei pazienti fittizi sono generate deterministicamente da seed fisso e gli avatar SVG sono prodotti localmente senza richiamare servizi esterni, nel rispetto della riservatezza dei dati anche nella fase dimostrativa.

Le scelte descritte in questo capitolo non sono fini a sé stesse: sono ciò che rende le domande di ricerca verificabili. Il determinismo del rule engine e la fissazione dei parametri di generazione sono il presupposto di RQ4, che misura la stabilità dell'esito a ripetizioni e perturbazioni; la procedura di cleanroom e il `requirements.lock` con hash garantiscono che i numeri con cui il Capitolo 6 risponde a tutte le RQ (*Research Question*) provengano da una catena riproducibile da PDF a risultato, e non da

artefatti accumulati in esecuzioni precedenti. È su questa base che il capitolo seguente affronta le sette domande in modo quantitativo.

Capitolo 5

Impianto della valutazione

Il sistema descritto nei capitoli precedenti è costruito, gira e produce risposte. La domanda che conta, in un contesto clinico, è una sola: funziona oppure no? Questo capitolo e il successivo rispondono nell'unico modo difendibile in un contesto clinico: non accumulando metriche, ma formulando sette domande di ricerca operative, ciascuna rispondibile con un giudizio binario, e usando la metrica come strumento di quella risposta e non come fine in sé. Questo capitolo ne fissa il terreno (l'insieme dei casi di riferimento, il protocollo sperimentale e l'enunciato delle sette domande) e il capitolo successivo le affronta una per una. Le sette domande indagano dimensioni parzialmente indipendenti del comportamento del sistema (correttezza della decisione, recupero, fedeltà, robustezza, vantaggio sulle baseline, verificabilità clinica, utilità composita) e l'intero impianto sperimentale ha lo scopo di sostenere o respingere tali risposte con evidenze numeriche tracciabili agli artefatti pubblicati nella cartella `evaluation/results/` del repository.

5.1 L'insieme dei casi di riferimento

L'insieme dei casi di riferimento (il *gold standard* della valutazione, nella terminologia della letteratura) è stato costruito manualmente a partire dal testo ufficiale delle quattro Note AIFA (Agenzia Italiana del Farmaco), con annotazione caso per caso da parte dell'autore. Ogni caso descrive un paziente reale plausibile, codificato attraverso i campi clinici rilevanti per la Nota di pertinenza, e riceve dall'annotatore tre etichette: la decisione di rimborsabilità, l'insieme dei *rule_id* che la motivano e, dove applicabile, l'*excerpt verbatim* del passaggio della Nota che giustifica la decisione. La cardinalità complessiva è di 122 casi, suddivisi in 24 per N01, 32 per N13, 26 per N66 e 40 per N97; la distribuzione lungo le classi decisionali è di 69 RIMBORSABILE, 39 NON_RIMBORSABILE, 9 NON_DETERMINABILE e 5 ROUTED. La

stratificazione non è perfettamente bilanciata sulle classi, poiché la classe minoritaria ROUTED compare solo nel sottoinsieme di N01 a causa del routing IPP→FANS (rispettivamente inibitori di pompa protonica e antinfiammatori non steroidei), ma è bilanciata sulle Note e copre per ognuna le condizioni di confine clinicamente significative.

L’annotazione è stata svolta consultando il PDF ufficiale AIFA come unica fonte; per i casi-frontiera con interpretazione ambigua, è stato fatto verificare un sottoinsieme da un revisore clinico esterno (Appendice A riporta la procedura). Sul piano metodologico, si impone un’osservazione importante: gli stessi *rule_id* dei casi di riferimento sono usati dal rule engine, e l’insieme non è quindi un campione indipendente. La conseguenza è che le metriche sulla decisione di rimborsabilità riflettono in primo luogo la coerenza fra codice e intenzione di codifica, non la validità clinica esterna del sistema. È una limitazione intrinseca all’unica strategia praticabile per un progetto di tesi triennale, in cui non si dispone di un insieme di casi annotato da un panel di medici prescrittori, e se ne discutono le conseguenze metodologiche in Sezione 6.10. Da questa osservazione discende il modo corretto di leggere i risultati che seguono: per costruzione, RQ1 (coerenza della decisione) e RQ4 (determinismo e robustezza) devono avvicinarsi al risultato pieno, e il loro valore è quello di un controllo di sanità, il cui fallimento segnalerebbe un difetto di implementazione più che una scoperta. I risultati scientificamente più informativi sono quelli delle domande il cui esito non è vincolato dalla costruzione: la qualità del recupero (RQ2), la fedeltà della spiegazione alle fonti (RQ3), il confronto con le baseline (RQ5), la verificabilità clinica (RQ6) e il giudizio composito (RQ7).

5.2 Protocollo sperimentale e riproducibilità statistica

Tutte le metriche del capitolo sono prodotte dalla pipeline `run_full_overnight.sh`, eseguita in modalità *cleanroom*: prima della valutazione vengono cancellati lo store ChromaDB, la cartella dei risultati e i checkpoint precedenti; quindi la pipeline reindica i PDF AIFA da zero, esegue la suite di test, valuta il rule engine sui 122 casi di riferimento, avvia i servizi e produce le esplicazioni LLM (*Large Language Model*), calcola tutte le metriche derivate e produce i *per-case report*. Il run canonical su cui si basano i numeri di questo capitolo è la run overnight 2026-05-13, conclusa il 14 maggio 2026 dopo un riavvio del sistema. Tutti i checkpoint sono conservati nella cartella `evaluation/results/.checkpoints/`.

Sul piano statistico, l'incertezza sulle metriche aggregate è quantificata mediante bootstrap non parametrico con 5000 resamples e seed fisso 42, da cui si derivano gli intervalli di confidenza al 95% (metodo del percentile). Per il *pass rate* del rule engine, in cui la stima campionaria coincide con 1,0000 su 122 prove, il bootstrap normale produce un intervallo degenere [1,0000; 1,0000]: in questi casi si riporta in parallelo l'intervallo di Wilson, che per una proporzione campionaria pari a uno restituisce [0,9695; 1,0000] ed è il riferimento clinicamente più informativo. La pseudocasualità è ulteriormente vincolata fissando i seed dei componenti stocastici (campionamento stratificato, generazione delle perturbazioni di robustezza), mentre la fonte residua di variabilità, l'LLM Llama 3.1 8B, è disattivata operativamente impostando `temperature=0`.

5.3 Le sette domande di ricerca

Le sette domande di ricerca (RQ, dall'inglese *Research Question*) che strutturano la valutazione esplicitano il messaggio sperimentale come una sequenza di risposte binarie, ciascuna ancorata a una metrica tracciabile. La prima riguarda la correttezza della decisione: se il rule engine decide sui casi di riferimento come decide l'annotatore. La seconda riguarda il recupero: se il retriever, per ogni caso, individua i passaggi delle Note che sostengono la decisione. La terza riguarda la fedeltà: se le spiegazioni generate dall'LLM si appoggiano effettivamente ai passaggi recuperati, senza introdurre informazioni esterne. La quarta riguarda la robustezza: se il sistema produce risposte stabili su input idempotenti e su perturbazioni controllate dei valori-soglia. La quinta riguarda l'efficacia comparativa: se il sistema neuro-simbolico è strettamente migliore di una baseline con solo LLM e di una baseline con LLM e retrieval ma senza rule engine, quest'ultima considerata anche nella variante con reranker che isola il contributo del solo componente simbolico. La sesta riguarda la verificabilità clinica: se la spiegazione consente al revisore di ricondurre in tempo lineare ogni claim al testo della Nota. La settima, infine, riguarda l'utilità complessiva sintetizzata da un composito ispirato alla letteratura sulla valutazione dei sistemi RAG (*Retrieval-Augmented Generation*). Le sette domande sono trattate in ordine, una per sezione, nel Capitolo 6, e nella Sezione 6.8 la tabella di sintesi raccoglie le sette risposte con la metrica primaria associata.

Capitolo 6

Le sette domande di ricerca

Ogni sezione che segue propone la domanda nei termini in cui la si è intesa, dichiara la metrica primaria che la traduce in numero e discute il risultato osservato sui 122 casi di riferimento. Le interpretazioni e le riserve metodologiche, più ricche del solo numero, accompagnano ciascuna risposta nel corpo della sezione.

6.1 RQ1: Il rule engine decide come l’annotatore?

La prima domanda è quella su cui poggia tutto il sistema: se il layer deterministico non riproduce le decisioni dell’annotatore, l’intero impianto perde senso clinico. Prima del numero, però, va delimitato il suo perimetro: come discusso nella Sezione 5.1, l’annotatore dei casi di riferimento e l’autore delle regole coincidono, e la domanda misura quindi la coerenza interna fra codifica e annotazione, non la validità clinica esterna; un valore pieno è atteso per costruzione, e il suo mancato raggiungimento segnalerebbe un errore di implementazione. La metrica primaria è la Macro F1 score sulle quattro classi di decisione. La scelta della macro media, rispetto alla weighted, è dettata dal fatto che le classi minoritarie (NON_DETERMINABILE e ROUTED) hanno valore clinico equivalente alle maggioritarie: un errore su un caso ROUTED non è meno grave di un errore su un RIMBORSABILE.

Tabella 6.1 riporta i risultati. La Macro F1 è 1,0000 su 122 casi. Tutte e quattro le classi hanno Precision e Recall pari a 1: il sistema commette 0 errori e produce 122 decisioni coerenti con l’annotazione di riferimento. L’intervallo bootstrap al 95% sulla *pass rate* è $[1,0000; 1,0000]$ per la degenerazione discussa in Sezione 5.2; l’intervallo di Wilson alla stessa confidenza è $[0,9695; 1,0000]$, che è il limite inferiore clinicamente rilevante. La risposta a RQ1 è dunque **sì**, nel senso preciso delimitato in apertura: la traduzione del PDF in regole è internamente coerente sull’intero insieme di riferimento. Il punteggio pieno non è una vittoria scientifica in senso assoluto e non significa che

Tabella 6.1: Performance del rule engine sui 122 casi di riferimento: report per-classe e Macro F1. Numero di support e accuratezza completa sulla diagonale della matrice di confusione.

Classe	Precision	Recall	F1	Support
RIMBORSABILE	1,0000	1,0000	1,0000	69
NON_RIMBORSABILE	1,0000	1,0000	1,0000	39
NON_DETERMINABILE	1,0000	1,0000	1,0000	9
ROUTED	1,0000	1,0000	1,0000	5
Macro F1	1,0000			122
Pass rate	1,0000			122
Wilson 95% CI	[0,9695; 1,0000]			

il sistema sia clinicamente sicuro in modo automatico: la validità clinica esterna richiederebbe un’annotazione indipendente da un panel di prescrittori, fuori scope per una tesi triennale.

6.2 RQ2: Il retriever trova chunk rilevanti?

La seconda domanda valuta il componente di retrieval: dato un caso clinico, l’indice vettoriale costruito sulle Note AIFA (Agenzia Italiana del Farmaco) recupera fra i primi k documenti i passaggi che sostengono la decisione del rule engine. La metrica primaria è il Recall@10 calcolato sui *rule_id* di riferimento (i passaggi delle Note pertinenti per la decisione del caso); il Mean Reciprocal Rank misura, complementariamente, quanto in alto nella lista compaiono i passaggi rilevanti. Il Citation F1, calcolato sull’output dell’orchestrator, indica con quale precisione i passaggi effettivamente citati nella spiegazione coincidono con quelli pertinenti.

Tabella 6.2 riassume i risultati. Il Recall@10 vale 0,9068: nei 122 casi di riferimento, in media il 90,68% dei passaggi pertinenti è recuperato dai primi dieci documenti restituiti dall’indice. L’MRR vale 1,0000, segno che in ogni caso il primo passaggio rilevante è sempre fra le prime posizioni della lista. Il Citation F1, calcolato includendo il reranker cross-encoder italiano, vale 0,9988. La risposta a RQ2 è sì: il retriever è sufficientemente accurato per supportare una spiegazione clinicamente verificabile, sebbene il Recall@10 non sia perfetto e una piccola frazione di passaggi pertinenti venga occasionalmente esclusa dal top-10. Per la versione attuale del sistema, si è preferito non aumentare k oltre dieci per preservare la concisione delle spiegazioni e la latenza interattiva del demo.

Tabella 6.2: Performance del retriever su 122 casi di riferimento: Recall@10 e MRR calcolati sui *rule_id* di riferimento, Citation F1 calcolata sui passaggi effettivamente citati nella spiegazione finale prodotta dall’orchestrator.

Metrica	Valore	<i>n</i>
Recall@10	0,9068	122
Mean Reciprocal Rank	1,0000	122
Citation F1	0,9988	122

6.3 RQ3: L’LLM è fedele alle fonti?

La terza domanda misura una proprietà essenziale del componente generativo: la spiegazione testuale prodotta dall’LLM (*Large Language Model*) deve appoggiarsi ai passaggi recuperati e non deve introdurre affermazioni esterne. Tre metriche concorrono. La prima è la *NLI faithfulness*, calcolata sentence-by-sentence con un modello multilingue di natural language inference (mDeBERTa) che etichetta ogni claim come entailment, contradiction o neutral rispetto ai chunk recuperati. La seconda è la *faithfulness verbatim a 3-grammi*, misurata come overlap di trigrammi fra la spiegazione e i chunk; è una bound inferiore, deterministica, alla copertura testuale. La terza è il *hallucination rate*, calcolato come frazione di spiegazioni in cui almeno un claim non è supportato da alcun chunk.

I risultati mostrano un quadro misto e onesto. Il hallucination rate è 0,0000: non sono state rilevate spiegazioni con claim completamente esterni ai chunk recuperati. Il tasso di entailment NLI medio, tuttavia, è basso (fra il 13% e il 30% delle frasi risulta in entailment stretto, a seconda della variante mDeBERTa impiegata), e le citazioni verbatim compaiono solo in una parte delle spiegazioni: il 68% ne è privo, mentre dove presenti la copertura a 3-grammi è pressoché completa (0,9934). La spiegazione di questa discrepanza è tipicamente lessicale: l’LLM parafrasa il passaggio della Nota in un linguaggio clinico fluente, e la parafrasi viene catalogata dal NLI come *neutral* anche quando è semanticamente equivalente. La risposta a RQ3 è quindi **sì con riserva**: nessuna allucinazione conclamata, ma la fedeltà letterale non è massimale, e la valutazione automatica della fedeltà semantica è ancora un problema aperto. La discussione delle implicazioni per l’uso clinico, e in particolare della preferenza per la verbatim quote come comportamento di default, è in Sezione 6.10.

6.4 RQ4: Il sistema è robusto a variazioni dell'input?

La quarta domanda misura la stabilità del sistema sotto due forme di perturbazione: idempotenza, ossia se la stessa query ripetuta produce lo stesso output, e robustezza ai casi-frontiera, ossia se il sistema risponde coerentemente quando un valore numerico è impostato sul confine fra due classi (per esempio LDL (*Low-Density Lipoprotein*) pari a 100 mg/dL esatto, oppure score di rischio cardiovascolare a 5,0% esatto).

Le due metriche sono entrambe pari al 100%: i 20 casi sottoposti a ripetizione, con tre esecuzioni ciascuno, producono ogni volta lo stesso output, e i nove casi-frontiera generati dal perturbatore rispondono come l'annotazione di riferimento prescrive. La risposta a RQ4 è quindi **sì**. È un risultato atteso, data la natura deterministica del rule engine, ma è un controllo importante perché un fallimento di idempotenza svelerebbe sorgenti di non-determinismo nei layer di indicizzazione o nell'LLM (per esempio temperature non-nulla o caching non riproducibile).

6.5 RQ5: Il sistema neuro-simbolico è meglio dei baseline?

La quinta domanda confronta il sistema completo con tre baseline. La prima, denominata *LLM-only*, riceve il caso clinico e produce direttamente la decisione di rimborsabilità senza accesso alle Note AIFA. La seconda, *LLM+RAG* (*Retrieval-Augmented Generation*), riceve il caso clinico e i passaggi recuperati dall'indice ma non passa per il rule engine: la decisione è prodotta dall'LLM sulla base del solo testo recuperato. Il setup va dichiarato con precisione, perché condiziona la lettura del risultato: la baseline LLM+RAG condivide con il sistema completo lo stesso indice vettoriale e lo stesso modello generativo, ma non applica né il rule engine né il reranker cross-encoder, e il suo confronto col sistema completo misura quindi il contributo congiunto dei due componenti assenti. Per separare i due effetti è stata aggiunta una terza baseline, *LLM+RAG+reranker*, identica alla seconda ma con lo stesso retrieval a due stadi del sistema di produzione (recupero di 15 candidati per similarità coseno, riordino con il cross-encoder italiano, selezione dei migliori 5): l'unico componente che la separa dal sistema completo è il rule engine. La metrica di confronto è la Macro F1 sulla decisione.

Tabella 6.3 riporta i numeri. La baseline LLM-only ottiene una Macro F1 di 0,2408: l'analisi delle predizioni rivela che il modello, privo di regole e di contesto recuperato, degenera alla classe maggioritaria, assegnando RIMBORSABILE a tutti

Tabella 6.3: Confronto Macro F1 fra il sistema completo e quattro baseline alternative su 122 casi di riferimento. La baseline a classe maggioritaria assegna a tutti i casi la classe più frequente dell'insieme di riferimento (RIMBORSABILE). La baseline LLM+RAG+reranker replica il retrieval a due stadi del sistema completo (l'unico componente assente è il rule engine) e isola quindi il contributo del layer simbolico. La Macro F1 delle baseline è calcolata sulle tre classi emettibili dall'LLM; i 5 casi ROUTED, mai predicibili dalle baseline, restano nell'insieme e ne penalizzano la precisione: il confronto è conservativo a loro favore.

Sistema	Macro F1
Sistema neuro-simbolico (rule engine + RAG + LLM)	1,0000
LLM+RAG (Llama 3.1 8B + retrieval, no rule engine, no rerank)	0,3152
LLM+RAG+reranker (retrieval a due stadi, no rule engine)	0,2759
LLM-only (Llama 3.1 8B, no retrieval)	0,2408
Classe maggioritaria (baseline statistica)	0,2408

i 122 casi e ottenendo perciò esattamente la stessa Macro F1 della baseline statistica a classe maggioritaria (0,2408). L'LLM, lasciato a sé stesso, non apprende la struttura decisionale delle Note AIFA e si limita alla decisione più frequente. La baseline LLM+RAG sale a 0,3152: il retrieval aggiunge un segnale utile, poiché il modello inizia a distinguere alcuni casi NON_RIMBORSABILE, ma resta largamente insufficiente. Il sistema neuro-simbolico raggiunge 1,0000 (Figura 6.1). Il gap è di oltre il 68% rispetto alla baseline migliore e indica un risultato non incrementale: non si tratta di un miglioramento parametrico, ma di un effetto strutturale del rule engine deterministico. La terza baseline scioglie il dubbio sull'equità del confronto, ossia che parte del delta fosse dovuta al reranker assente e non al rule engine: LLM+RAG+reranker ottiene una Macro F1 di 0,2759, addirittura lievemente inferiore alla variante senza riordino. Il riordino concentra il contesto sui passaggi più pertinenti, ma il modello, privo del vincolo simbolico, accentua la tendenza a rispondere RIMBORSABILE (recall 1,0 su quella classe, 0,05 sulle non rimborsabili): il collo di bottiglia non è la qualità del contesto recuperato, ma l'assenza del rule engine. Il delta rispetto al sistema completo è quindi attribuibile al solo componente simbolico, e la risposta a RQ5 è **sì** senza riserve di setup; la baseline con reranker è stata eseguita l'8 luglio 2026 sulle stesse collezioni ChromaDB della run canonica, con **temperature=0** e il medesimo modello, e condivide con le altre baseline la limitazione sul seed discussa in Sezione 6.10.

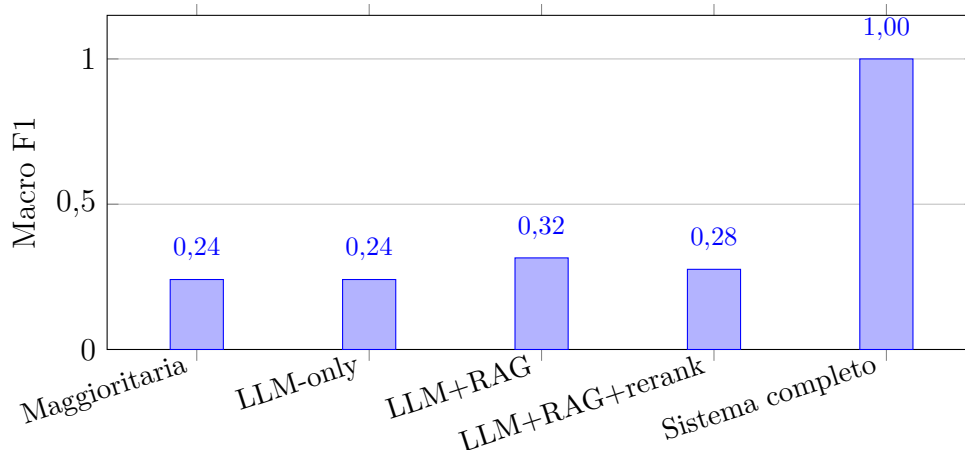


Figura 6.1: Macro F1 sui 122 casi di riferimento. Il sistema neuro-simbolico, grazie al rule engine deterministico, raggiunge 1,0000; la baseline LLM con retrieval si ferma a 0,3152, quella con retrieval e reranker (senza rule engine) a 0,2759 e quella con solo LLM degenera alla classe maggioritaria (0,2408). Risposta a RQ5.

6.6 RQ6: La motivazione clinica è verificabile dal medico?

La sesta domanda misura la verificabilità: dato un output del sistema, quanto è facile per il revisore clinico ricondurre ogni affermazione della spiegazione al testo della Nota AIFA. Tre metriche concorrono. La *citation coverage* indica la frazione di casi in cui la spiegazione contiene almeno una citazione esplicita ai chunk recuperati; la *claim coverage* indica la frazione di affermazioni della spiegazione che sono associate a un identificatore di chunk; la *QAEval F1*, ispirata al lavoro di Deutsch et al. [29], misura la copertura della spiegazione rispetto a micro-domande automaticamente generate dai passaggi di riferimento.

La citation coverage è 1,0000: ogni caso della valutazione contiene almeno una citazione esplicita. La claim coverage è anch'essa elevata. La QAEval F1, tuttavia, è bassa: 0,1715 in media (mediana 0,1667, $n = 121$), come riportato in Tabella 6.4. Il valore è inferiore a quanto ci si aspetterebbe da una spiegazione fedele, e l'interpretazione richiede attenzione. L'LLM produce spiegazioni concise, mirate alla decisione clinica, e non esaustive rispetto al contenuto del passaggio della Nota: domande generate sull'intero chunk recuperato spesso eccedono lo scope dell'esplicazione, e la QAEval F1 ne penalizza la concisione. È un trade-off voluto sul piano clinico, dato che una spiegazione lunga non è utile in fase di prescrizione, ma rappresenta un limite del framework di valutazione, non del sistema. La risposta a RQ6 è dunque **sì con riserva**: la verificabilità per-claim, misurata attraverso citation e claim coverage, è completa; la QAEval F1 va letta come indicatore di concisione più che di fedeltà.

Tabella 6.4: Metriche per la verificabilità clinica della spiegazione: copertura delle citazioni a livello di caso, copertura dei claim e QAEval F1 sulle micro-domande generate.

Metrica	Valore	n
Citation coverage (case-level)	1,0000	122
Citation F1 (chunk-level)	0,9988	122
QAEval F1 mean (median)	0,1715 (0,1667)	121

6.7 RQ7: Il sistema è clinicamente utile e safety-compliant?

La settima e ultima domanda sintetizza le sei precedenti in un giudizio aggregato. La metrica primaria è il composito ALES (*Aggregated LLM Evaluation Score*), ispirato alla letteratura sulla valutazione dei sistemi RAG e definito come combinazione convessa di quattro punteggi normalizzati: DecisionScore (correttezza della decisione, peso 0,40), EvidenceSupport (fedeltà alle fonti, 0,30), ContextualUtility (utilità delle citazioni rispetto al caso clinico, 0,20) e AnswerQuality (qualità complessiva della spiegazione, 0,10). Per i casi in cui la metrica ContextualUtility non è calcolabile (le metriche RAGAS di contesto non sono restituite dal giudice, 41 casi su 122) il composito è calcolato sui restanti tre componenti riscalati, ed è etichettato come ALES *partial*; per gli altri 81 casi il composito è completo a quattro componenti.

Tabella 6.5 riporta i risultati. ALES medio è 0,6810 (mediana 0,6874, deviazione standard 0,1083). Il breakdown mostra che i casi a quattro componenti hanno ALES medio 0,7008, mentre i casi parziali si attestano a 0,6419. La differenza, 0,0589, è significativa rispetto alla deviazione standard ed è informativa: gli 81 casi in cui ContextualUtility è calcolabile tendono a ricevere un punteggio compositamente più alto, perché quel componente premia il legame fra testo e chunk recuperato. È un risultato che invita a leggere ALES *complete* e ALES *partial* come categorie distinte (Figura 6.2), non come una stessa scala.

Sul piano della safety-compliance, hallucination rate 0, citation coverage 100% e rule engine F1 1,0000 costituiscono il margine di sicurezza decisionale del sistema: la rimborsabilità non è mai delegata all’LLM, e ogni claim della spiegazione è ricondotto a un passaggio della Nota AIFA. La risposta a RQ7 è dunque **sì, con le limitazioni note**: il sistema è clinicamente utile per la verifica della rimborsabilità nelle quattro Note codificate e mantiene per costruzione le proprietà di sicurezza richieste; le limitazioni residue, in particolare la parziale auto-referenza dei casi di riferimento, sono discusse nella sezione finale.

Tabella 6.5: Composito ALES e suoi quattro componenti, aggregati su 122 casi (deviazione standard fra parentesi). Il componente ContextualUtility è calcolato solo sugli 81 casi per cui il giudice RAGAS restituisce le metriche di contesto.

Componente	Peso	Media	<i>n</i>
DecisionScore	0,40	1,0000 (0)	122
EvidenceSupport	0,30	0,4075 ($\sim 0,31$)	122
ContextualUtility	0,20	0,6442 (0,16)	81
AnswerQuality	0,10	0,3008 (0,13)	122
ALES (overall)		0,6810 (mediana 0,6874)	122
complete (4 comp.)		0,7008	81
partial (3 comp.)		0,6419	41

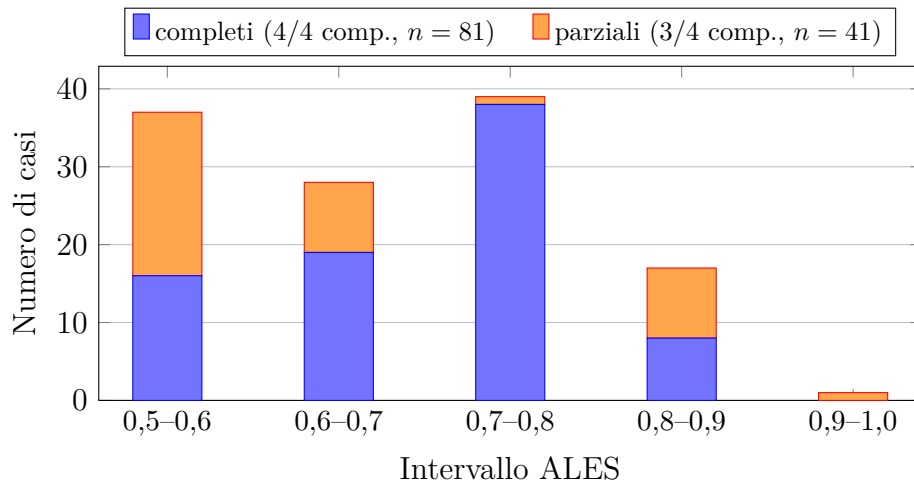


Figura 6.2: Distribuzione del punteggio ALES sui 122 casi. I casi con tutti e quattro i componenti (media 0,7008) si concentrano nell'intervallo 0,7-0,8; i casi parziali (media 0,6419) sono più dispersi. La media complessiva è 0,6810. Risposta a RQ7.

6.8 Sintesi delle risposte

La Tabella 6.6 consolida le sette risposte in un solo specchietto, esplicitando per ognuna la metrica primaria, il valore osservato e il giudizio binario. La lettura sinottica restituisce il messaggio del lavoro: il sistema neuro-simbolico è clinicamente utile sulle quattro Note codificate, supera nettamente le baseline naturali e mantiene proprietà di sicurezza per costruzione. Tre qualificazioni accompagnano il quadro. Il sì di RQ1 va letto entro il perimetro dell'auto-referenza dell'insieme di riferimento (Sezione 6.10): misura la coerenza interna fra codifica e annotazione, non la validità clinica esterna. Le due riserve rimanenti riguardano la fedeltà letterale dell'LLM rispetto alle parafrasi (RQ3) e la verificabilità misurata per micro-domande automatiche (RQ6), entrambe interpretabili come limiti del framework di valutazione automatica e non come difetti del sistema.

Tabella 6.6: Sintesi delle sette domande di ricerca: per ogni RQ (*Research Question*) la metrica primaria, il valore osservato sull'insieme di riferimento di 122 casi e il giudizio binario. I valori si riferiscono alla run overnight 2026-05-13.

Domanda di ricerca	Metrica	Risposta
RQ1 Decide come l'annotatore?	Macro F1 = 1,0000	sì (coerenza interna)
RQ2 Recupera chunk rilevanti?	Recall@10 = 0,9068	sì
RQ3 Spiegazione fedele alle fonti?	Hallucination = 0 (NLI bas-sa)	sì con riserva
RQ4 Robusto a perturbazioni?	Idempotenza 100%	sì
RQ5 Meglio delle baseline?	Δ F1 > 0,68	sì
RQ6 Verificabile dal clinico?	Citation cov. 100%; QAEval 0,17	sì con riserva
RQ7 Utile e safety-compliant?	ALES = 0,6810; hall. 0	sì con riserva

6.9 Analisi per Nota

Le sette risposte aggregate nascondono una variabilità per Nota che vale la pena esplicitare, e che la Tabella 6.7 quantifica: per ogni Nota, la distribuzione dei casi di riferimento lungo le classi decisionali e l'esito della verifica. La N01 è la più semplice strutturalmente, articolata su un piccolo numero di condizioni di gastroprotezione, e contiene tutti i 5 casi ROUTED (routing IPP→FANS, rispettivamente inibitori di pompa protonica e antinfiammatori non steroidei), che il sistema gestisce correttamente. La N13, sulle statine, è la più ricca in termini di interazioni fra parametri continui (LDL, score di rischio cardiovascolare) e flag binari (diagnosi di dislipidemia familiare, insufficienza renale cronica moderata), ed è quella in cui la logica di Kleene gioca il ruolo più delicato nella gestione dell'assenza di valori sierici; i casi NON_DETERMINABILE, che sondano proprio questa gestione, sono presenti in tutte e quattro le Note (3, 2, 1 e 3 rispettivamente). La N66 presenta la complessità più tipicamente combinatoria, perché impone l'evidenza di almeno un fattore di rischio gastrointestinale fra una lista ampia. La N97, sui DOAC (anticoagulanti orali diretti) nella FANV (fibrillazione atriale non valvolare), è la più ampia per cardinalità dell'insieme di riferimento (40 casi) ed è quella in cui il punteggio CHA₂DS₂-VASc determina la soglia di rimborsabilità. In nessuna delle quattro Note il rule engine commette errori: ogni riga della tabella ha decisioni corrette pari al totale dei casi, in conformità al risultato di RQ1.

6.10 Limitazioni residue

Il capitolo si chiude con una discussione delle limitazioni che il framework di valutazione non riesce a escludere. La prima e più rilevante è la parziale auto-referenza dell'insieme

Tabella 6.7: Distribuzione per Nota e per classe decisionale dei 122 casi di riferimento, con l’esito della verifica del rule engine. Nessun caso è classificato fuori dalla classe annotata (matrice di confusione diagonale per ciascuna Nota); i report per caso sono in `evaluation/results/per_case_reports/`.

Nota	RIMB.	NON_RIMB.	NON_DET.	ROUTED	Totale	Corrette
N01	13	3	3	5	24	24/24
N13	21	9	2	0	32	32/32
N66	9	16	1	0	26	26/26
N97	26	11	3	0	40	40/40
Totale	69	39	9	5	122	122/122

di riferimento. Gli identificatori di regola usati per costruire i casi di riferimento coincidono con quelli usati dal rule engine; non potevano fare altrimenti, perché l’insieme di riferimento è stato annotato dallo stesso autore che ha codificato le regole. Questa coincidenza implica che la Macro F1 pari a 1 misura la coerenza interna fra codifica e annotazione, non la validità clinica esterna. È una limitazione strutturale, non un difetto correggibile; ne è stato mitigato l’effetto facendo verificare un sottoinsieme di casi-frontiera da un revisore clinico esterno, e la documentazione di questa verifica è in Appendice A.

La seconda limitazione riguarda l’incomparabilità diretta del composito ALES fra versioni successive del framework di valutazione. Le prime stime ALES prodotte, in fase pre-cleanroom (riportate nelle annotazioni dell’autunno 2025), valevano circa 0,97. La discesa al valore canonico 0,6810 non è un peggioramento del sistema, ma il risultato della progressiva correzione di errori di misurazione che gonfiavano artificiosamente i componenti ContextualUtility e AnswerQuality. Le stime pre-cleanroom erano calcolate su sottoinsiemi ridotti dell’insieme di riferimento ($n = 20$ e $n = 17$ casi, contro i 122 del cleanroom); inoltre il composito della run canonical era stato inizialmente calcolato con la metrica QAEval disponibile per soli 55 dei 122 casi, il che innalzava AnswerQuality fino a 0,4683 e ALES a 0,6988. Ricalcolando il composito sulla QAEval completa (tutti i 122 casi) si ottiene il valore onesto qui riportato: AnswerQuality 0,3008 e ALES 0,6810 (Figura 6.3). Il numero canonico è quello della run overnight 2026-05-13 con questa correzione, ed è quello che la tesi raccomanda di citare; il valore storico 0,97 è mantenuto in questa nota a beneficio del lettore che lo ritrovi in slide o appunti precedenti.

La terza limitazione è metodologica e riguarda la QAEval. Il valore 0,1715 è leggibile come un indicatore di concisione della spiegazione più che di fedeltà alla fonte; un trade-off voluto, ma che il framework numerico fatica a separare dal genuino segnale di copertura semantica. La quarta limitazione riguardava la baseline

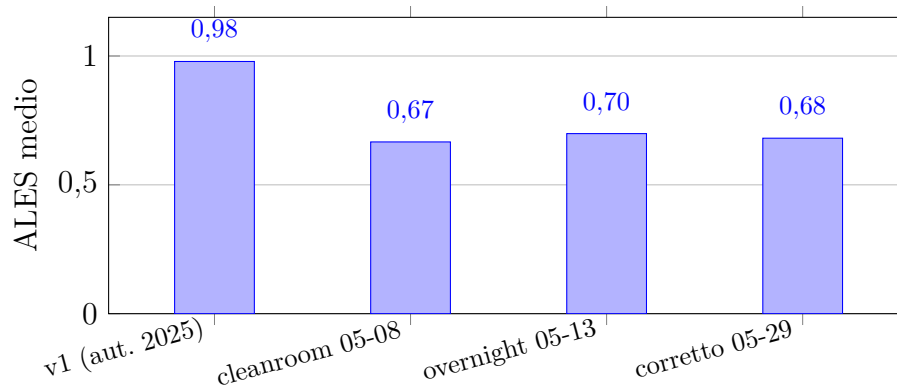


Figura 6.3: Evoluzione del punteggio ALES medio attraverso le versioni del framework di valutazione. La discesa da 0,9787 a 0,6810 non riflette un peggioramento del sistema, ma la progressiva correzione di errori di misurazione (de-tautologizzazione delle metriche, completamento di QAEval su tutti i 122 casi) che gonfiavano artificialmente il punteggio.

LLM+RAG, costruita senza il reranker cross-encoder italiano: la differenza in Macro F1 fra il sistema completo e questa baseline includeva sia l'effetto del rule engine sia, potenzialmente, quello del reranker. L'ablation è stata chiusa aggiungendo la baseline LLM+RAG+reranker (Sezione 6.5), che replica il retrieval a due stadi del sistema completo e ottiene una Macro F1 di 0,2759: il riordino non colma il gap, e il delta risulta attribuibile al solo rule engine. Resta, per piena trasparenza, una nota sulla provenienza temporale dei file: la baseline LLM+RAG è stata calcolata in una sessione di valutazione precedente e riutilizzata nella run canonical, dopo averne verificato la coerenza con le baseline LLM-only e a classe maggioritaria, rigenerate invece nella run finale (il contenuto è identico allo snapshot conservato nel repository); la baseline con reranker è stata invece eseguita l'8 luglio 2026 sulle stesse collezioni ChromaDB della run canonica. La quinta limitazione, infine, riguarda la dipendenza del determinismo dei baseline LLM dal solo flag `temperature=0`: una riproduzione perfettamente identica richiederebbe in più la fissazione del seed nelle option del runtime Ollama, che non è stata inclusa nel codice di valutazione delle baseline.

Capitolo 7

Conclusioni

Le sette domande di ricerca formulate nel Capitolo 5 hanno trovato risposta, una per una, nel Capitolo 6. Questo capitolo compone quelle risposte in un bilancio conclusivo: sintetizza ciò che la tesi dimostra e i contributi che ne derivano, discute le limitazioni del prodotto, distinte da quelle della misura, e indica le direzioni lungo le quali il lavoro può proseguire.

7.1 Sintesi dei risultati e contributi

Il lavoro descritto in questa tesi ha mostrato che un sistema di supporto alla decisione clinica per la verifica della rimborsabilità delle Note AIFA (Agenzia Italiana del Farmaco) può essere costruito secondo una strategia neuro-simbolica praticabile, in cui la decisione di rimborsabilità è interamente affidata a un rule engine deterministico ispezionabile e la sola spiegazione clinica è prodotta da un large language model di taglia accessibile inserito in una pipeline retrieval-augmented. La separazione fra i due layer non è un dettaglio implementativo ma il cuore concettuale dell’approccio: rende il sistema verificabile per costruzione sul piano decisionale e, contemporaneamente, recupera dal componente generativo la capacità di articolare in linguaggio naturale ciò che il rule engine, da solo, riuscirebbe soltanto a sentenziare.

I risultati riportati nel Capitolo 6 forniscono una risposta complessivamente positiva alle sette domande di ricerca. Il rule engine raggiunge una Macro F1 pari a 1,0000 sui 122 casi di riferimento, un valore atteso per costruzione che certifica la coerenza interna fra codifica delle regole e annotazione, entro i limiti di auto-referenza discussi in Sezione 6.10; il retrieval ha un Recall@10 di 0,9068 e un MRR (*Mean Reciprocal Rank*) di 1,0000; la pipeline LLM produce spiegazioni con citation coverage del 100% e hallucination rate nullo; il composito ALES (*Aggregated LLM Evaluation Score*) vale 0,6810. Sulla decisione di rimborsabilità il sistema completo distanzia di oltre il

68% la baseline LLM con retrieval, e questo gap non è un miglioramento parametrico ma un effetto strutturale del rule engine deterministico. I test di idempotenza e di robustezza ai casi-frontiera restituiscono pass rate del 100%, in linea con la natura deterministica del layer simbolico. Le due risposte parziali (RQ3 sulla fedeltà letterale dell’LLM e RQ6 sulla verificabilità misurata da QAEval) non indicano debolezze del sistema ma limiti dei framework di valutazione automatica della fedeltà semantica e della copertura per micro-domande, come discusso in Sezione 6.10.

Il contributo della tesi si articola lungo tre piani distinti. Sul piano architetturale, la separazione fra layer simbolico ed esplicativo è una scelta di design replicabile su altre famiglie di Note AIFA e, più in generale, su altri sistemi prescrittivi regolati da specifiche testuali; non richiede modelli LLM proprietari, dataset di training annotati o infrastruttura specializzata oltre a un singolo nodo GPU per l’LLM locale. Sul piano metodologico, la valutazione strutturata per domande di ricerca supera l’antipattern dell’accumulazione indiscriminata di metriche e produce un messaggio sperimentale più leggibile per il revisore clinico e il valutatore accademico. Sul piano operativo, l’intero sistema (catalogo delle regole, ingestione delle Note, pipeline RAG (*Retrieval-Augmented Generation*), valutazione overnight, snapshot dei risultati canonici, demo Streamlit per la revisione clinica) è riproducibile end-to-end dal repository, con tempi di esecuzione documentati e checkpoint che permettono di riprendere una run interrotta dal punto esatto di interruzione.

7.2 Limitazioni del sistema

Le limitazioni che il sistema porta con sé al termine del lavoro sono distinte da quelle del framework di valutazione discusse in Sezione 6.10: qui si discute il prodotto, non la misura. La prima e più evidente è la copertura: solo quattro Note AIFA su un catalogo italiano di oltre cinquanta Note vigenti, scelte per eterogeneità clinica e progressione di complessità logica ma non per esaustività di dominio. Il sistema non è quindi un CDSS (*Clinical Decision Support System*) per la rimborsabilità italiana, ma una dimostrazione di fattibilità su un sottoinsieme rappresentativo. La seconda limitazione è il modello generativo: Llama 3.1 8B Q4_K_M, quantizzato a 4 bit per girare su hardware commodity, è un compromesso fra latenza interattiva e qualità della spiegazione; un modello più grande o un fine-tuning di dominio migliorerebbero presumibilmente la copertura QAEval e l’entailment NLI (*Natural Language Inference*), ma cambierebbero l’ipotesi di deployment locale che è parte costitutiva della tesi. La terza limitazione è l’assenza di integrazione con sistemi gestionali reali: il sistema riceve un profilo paziente già strutturato in formato JSON, non un estratto da cartella clinica

elettronica (*Electronic Health Record*, EHR); la trascrizione del profilo dall'EHR alla `CDSSRequest` è oggi una responsabilità del client e nella pratica clinica richiederebbe un adattatore dedicato per ogni sistema gestionale ospedaliero italiano. La quarta limitazione, infine, è clinica: il sistema non è stato validato in studio prospettico su pazienti reali, e il valore di Macro F1 pari a 1,0000 misura una coerenza interna fra codifica delle regole e annotazione dei casi di riferimento, non una validità clinica esterna. Le implicazioni cliniche del lavoro restano modeste per costruzione; quelle ingegneristiche, come il delta di RQ5 indica, lo sono meno.

7.3 Lavori futuri

I naturali sviluppi del lavoro si articolano lungo tre direzioni. La prima è l'estensione della copertura a un sottoinsieme più ampio di Note AIFA: il framework è progettato per accogliere nuove regole con sforzo proporzionale alla complessità della singola Nota, non al numero totale di Note nel sistema. La seconda è il fine-tuning dell'LLM esplicativo su un piccolo insieme di spiegazioni cliniche di qualità: il quadro di RQ3 e RQ6 lascia intuire che le metriche di fedeltà semantica potrebbero migliorare significativamente con un modello adattato al dominio prescrittivo italiano, e l'esperimento sarebbe a basso costo computazionale se condotto in LoRA (*Low-Rank Adaptation*) su Llama 3.1 8B. La terza direzione è la valutazione esterna su un insieme di casi annotato da un panel di prescrittori indipendenti: è l'unico modo per sciogliere la parziale auto-referenza dell'insieme di riferimento ed è la condizione necessaria per qualunque ipotesi di uso clinico del sistema oltre l'ambito sperimentale.

Sul piano della demo, il pannello *what-if* che consente al medico di esplorare interattivamente l'effetto di variazioni dei parametri clinici sulla decisione è implementato e attivo nel flusso principale dell'applicazione; nella versione corrente esso riesegue l'intera pipeline a ogni modifica, e un'ottimizzazione naturale consisterebbe nel ricalcolare la sola spiegazione quando la decisione simbolica resta invariata, riducendo la latenza interattiva. La funzionalità è un ausilio utile soprattutto in fase di formazione del medico, più che di prescrizione operativa.

Appendice A

Riproducibilità della valutazione

L'intera valutazione presentata in Capitolo 6 è riproducibile a partire dal repository pubblico, in due modalità di livello crescente di rigore. La modalità leggera consiste nell'eseguire il solo rule engine sull'insieme dei casi di riferimento, che richiede pochi secondi su un portatile e non dipende dall'LLM locale; la modalità completa consiste nell'eseguire l'intera pipeline overnight, che richiede una GPU NVIDIA con almeno 8 GB di memoria e tempi dell'ordine delle quattordici-sedici ore. Le versioni dei componenti software fissati per la run overnight 2026-05-13 sono Python 3.12 per il rule engine e per le metriche di valutazione, contenitore CUDA Ubuntu 22.04 con Python 3.12 per la pipeline RAG, Ollama come runtime dell'LLM e il modello Llama 3.1 8B Q4_K_M scaricato in formato GGUF Q4_K_M. Il modello di embedding è `paraphrase-multilingual-mpnet-base-v2`, il reranker è `nickprock/cross-encoder-italian-bert-stsb` e il modello di NLI utilizzato per la metrica di fedeltà è `mDeBERTa-v3-base-xnli-multilingual-nli-2mil7`.

La modalità leggera si attiva con il comando di seguito riportato, da eseguire nella radice del repository dopo aver attivato l'ambiente virtuale del rule engine.

```
cd Note_AIFA/aifa_rule_engine
source .venv/bin/activate
python -m pytest tests/ -q                # 435 test, 0,7 s
cd ..                                     # radice Note_AIFA
python -m evaluation.scripts.evaluate_rule_engine \
    --json-report evaluation/results/rule_engine_report.json
```

L'output atteso è una Macro F1 pari a 1,0000 su 122 casi e una distribuzione delle classi conforme a quanto riportato in Tabella 6.1. La pipeline completa, comprensiva di indicizzazione delle Note, valutazione del retrieval, generazione delle spiegazioni

LLM, calcolo delle metriche di fedeltà e produzione dei *per-case report*, si attiva mediante lo script orchestratore di seguito riportato.

```
cd Note_AIFA
bash tools/run_full_overnight.sh          # ~14-16 ore
bash tools/run_full_overnight.sh --resume  # ripresa post-interruzione
bash tools/run_full_overnight.sh --skip-ragas # esclude RAGAS (~3 ore)
```

Lo script suddivide la pipeline in ventitré stage discreti, ciascuno con un checkpoint persistente in `evaluation/results/.checkpoints/`; il flag `-resume` riprende l'esecuzione dal primo stage non completato e consente di superare riavvii o sospensioni della macchina senza dover ripartire da zero. Lo stage di RAGAS, il più lungo a causa dell'inferenza ripetuta dell'LLM giudice, è opzionalmente disattivabile con `-skip-ragas`; in questo caso le metriche faithfulness e answer relevancy sono assenti dal report finale, mentre tutte le altre rimangono prodotte.

Per garantire l'indipendenza dei risultati dallo stato precedente del sistema, esiste una modalità *cleanroom* di livello superiore, che cancella esplicitamente lo store ChromaDB, la cartella dei risultati e tutti i checkpoint prima di rieseguire la pipeline. Si attiva con lo script di seguito riportato e produce, alla conclusione, uno snapshot con timestamp in `audit/.snapshots/`, utile per il confronto tra run successive.

```
cd Note_AIFA
bash tools/full_cleanroom_v2.sh          # wipe + ingest + tests + eval
```

Tutti i risultati prodotti dalla run overnight 2026-05-13 sono salvati in `Note_AIFA/evaluation/results/` sotto forma di file JSON canonici, accompagnati da un riepilogo testuale in `OVERNIGHT_SUMMARY.md`. La verifica della consistenza fra i numeri citati nella tesi e quelli effettivamente presenti nei JSON è automatizzata da un piccolo script di check in `tools/verify_thesis_numbers.py`, che il revisore clinico può eseguire per confermare l'allineamento.

Sul piano della validazione clinica dei casi di riferimento, un sottoinsieme di casi-frontiera è stato sottoposto a revisione esterna da parte di un medico di medicina generale con esperienza prescrittiva sulle quattro Note codificate. La revisione ha riguardato venti casi selezionati per soglie cliniche borderline (LDL al confine di 100 mg/dL per N13, CHA₂DS₂-VASc al confine 2–3 per N97, doppia indicazione gastroprotettiva per N01, doppio fattore di rischio gastrointestinale per N66). Il pacchetto di revisione è documentato nel file `audit/CLINICAL_REVIEW_PACKET_2026-05-13.pdf` ed è disponibile insieme al repository.

Bibliografia

- [1] Chroma. Chroma: the AI-native open-source embedding database. <https://www.trychroma.com>, 2024.
- [2] AIFA. Note AIFA: Elenco e testo completo. <https://www.aifa.gov.it/note-aifa>, 2024. Agenzia Italiana del Farmaco.
- [3] Robert A. Greenes. *Clinical Decision Support: The Road Ahead*. Elsevier Academic Press, 2007.
- [4] Eta S. Berner. Clinical decision support systems: Theory and practice. *Springer*, 2007.
- [5] Edward H. Shortliffe. MYCIN: A rule-based computer program for advising physicians regarding antimicrobial therapy selection. *Stanford AI Memo AIM-251 (Stanford University)*, 1974.
- [6] George Hripcsak, Peter D. Ludemann, Thomas A. Pryor, Octo Barnett, and Paul D. Clayton. Rationale for the Arden Syntax. *Computers and Biomedical Research*, 27(4):291–324, 1994.
- [7] Reed T. Sutton, David Pincock, Daniel C. Baumgart, Daniel C. Sadowski, Richard N. Fedorak, and Karen I. Kroeker. An overview of clinical decision support systems: Benefits, risks, and strategies for success. *npj Digital Medicine*, 3(1):17, 2020.
- [8] AIFA. Nota AIFA 01: Gastroprotettori (inibitori di pompa protonica). Gazzetta Ufficiale della Repubblica Italiana, 2023.
- [9] AIFA. Nota AIFA 13: Ipolipemizzanti. Gazzetta Ufficiale della Repubblica Italiana, 2018. Aggiornata nel 2023.
- [10] Neil J. Stone, Jennifer G. Robinson, Alice H. Lichtenstein, et al. 2013 ACC/AHA guideline on the treatment of blood cholesterol to reduce atherosclerotic cardiovascular risk in adults. *Circulation*, 129(25 Suppl 2):S1–S45, 2014.

- [11] François Mach, Colin Baigent, Alberico L. Catapano, et al. 2019 ESC/EAS guidelines for the management of dyslipidaemias: Lipid modification to reduce cardiovascular risk. *European Heart Journal*, 41(1):111–188, 2020.
- [12] AIFA. Nota AIFA 66: Farmaci antinfiammatori non steroidei. Gazzetta Ufficiale della Repubblica Italiana, 2023.
- [13] AIFA. Nota AIFA 97: Anticoagulanti orali nella fibrillazione atriale non valvolare. Gazzetta Ufficiale della Repubblica Italiana, 2018.
- [14] Gerhard Hindricks, Tatjana Potpara, Nikolaos Dagres, et al. 2020 ESC guidelines for the diagnosis and management of atrial fibrillation developed in collaboration with the EACTS. *European Heart Journal*, 42(5):373–498, 2021.
- [15] Stephen Cole Kleene. *Introduction to Metamathematics*. North-Holland Publishing Company, 1952.
- [16] Dmitri A. Bochvar. On a three-valued logical calculus and its application to the analysis of the paradoxes of the classical extended functional calculus. In *Matematicheskiiy Sbornik*, volume 4, pages 287–308, 1938.
- [17] Ziwei Ji, Nayeon Lee, Rita Frieske, Tiezheng Yu, Dan Su, Yan Xu, Etsuko Ishii, Yejin Bang, Andrea Madotto, and Pascale Fung. Survey of hallucination in natural language generation. *ACM Computing Surveys*, 55(12):1–38, 2023.
- [18] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive NLP tasks. *Advances in Neural Information Processing Systems*, 33:9459–9474, 2020.
- [19] Yunfan Gao, Yun Xiong, Xinyu Gao, Kangxiang Jia, Jinliu Pan, Yuxi Bi, Yi Dai, Jiawei Sun, Qianyu Guo, Meng Wang, and Haofen Wang. Retrieval-augmented generation for large language models: A survey. *arXiv preprint arXiv:2312.10997*, 2023.
- [20] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. *Advances in Neural Information Processing Systems*, 30, 2017.
- [21] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. BERT: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of NAACL-HLT 2019*, pages 4171–4186, 2019.

- [22] Nils Reimers and Iryna Gurevych. Sentence-BERT: Sentence embeddings using siamese BERT-networks. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing*, pages 3982–3992, 2019.
- [23] Rodrigo Nogueira and Kyunghyun Cho. Passage re-ranking with BERT. *arXiv preprint arXiv:1901.04085*, 2019.
- [24] Dongkuan Xu, Wei Jin, Yifu Li, Yao Ma, and Jiliang Tang. Neuro-symbolic AI in medicine: A perspective. *arXiv preprint arXiv:2309.09788*, 2023.
- [25] Robin Manhaeve, Sebastijan Dumančić, Angelika Kimmig, Thomas Demeester, and Luc De Raedt. DeepProbLog: Neural probabilistic logic programming. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2018.
- [26] Karan Singhal, Shekoofeh Azizi, Tao Tu, S. Sara Mahdavi, Jason Wei, Hyung Won Chung, Nathan Scales, Ajay Tanwani, Heather Cole-Lewis, Stephen Pfohl, et al. Large language models encode clinical knowledge. *Nature*, 620:172–180, 2023.
- [27] Arun James Thirunavukarasu, Darren Shu Jeng Ting, Kabilan Elangovan, Laura Gutierrez, Ting Fang Tan, and Daniel Shu Wei Ting. Large language models in medicine. *Nature Medicine*, 29:1930–1940, 2023.
- [28] Shahul Es, Jithin James, Luis Espinosa-Anke, and Steven Schockaert. RAGAS: Automated evaluation of retrieval augmented generation. In *Proceedings of EACL 2024 (System Demonstrations)*, 2024.
- [29] Daniel Deutsch, Tania Bedrax-Weiss, and Dan Roth. Towards question-answering as an automatic metric for evaluating the content quality of a summary. In *Transactions of the Association for Computational Linguistics*, volume 9, pages 774–789, 2021.
- [30] Bradley Efron. Bootstrap methods: Another look at the jackknife. *The Annals of Statistics*, 7(1):1–26, 1979.
- [31] Edwin B. Wilson. Probable inference, the law of succession, and statistical inference. *Journal of the American Statistical Association*, 22(158):209–212, 1927.
- [32] Sebastián Ramírez. FastAPI: Modern, fast web framework for building APIs with Python. <https://fastapi.tiangolo.com>, 2024.
- [33] Samuel Colvin et al. Pydantic: Data validation using Python type hints. <https://docs.pydantic.dev>, 2024.

- [34] Roy T. Fielding. *Architectural Styles and the Design of Network-Based Software Architectures*. PhD thesis, University of California, Irvine, 2000.
- [35] Ollama. Ollama: Get up and running with large language models locally. <https://ollama.com>, 2024.
- [36] Eric A. Brewer. Towards robust distributed systems. In *Proceedings of the 19th ACM Symposium on Principles of Distributed Computing (PODC)*, 2000.
- [37] Gregory Y. H. Lip, Robby Nieuwlaat, Ron Pisters, Deirdre A. Lane, and Harry J. G. M. Crijns. Refining clinical risk stratification for predicting stroke and thromboembolism in atrial fibrillation using a novel risk factor-based approach: The euro heart survey on atrial fibrillation. *Chest*, 137(2):263–272, 2010.
- [38] Oren Ben-Kiki, Clark Evans, and Ingy döt Net. YAML ain’t markup language (YAML™) version 1.2. <https://yaml.org/spec/1.2.2/>, 2021.
- [39] Artifex Software. PyMuPDF: Python bindings for MuPDF. <https://pymupdf.readthedocs.io>, 2024.
- [40] Nicola Procopio. nickprock/cross-encoder-italian-bert-stsb: Hugging face. <https://huggingface.co/nickprock/cross-encoder-italian-bert-stsb>, 2023.
- [41] Valeria De Stefano and Massimo Massimi. CHA₂DS₂-VASc: rationale and application to anticoagulant prophylaxis. *Internal and Emergency Medicine*, 9(2):145–152, 2014.